From Policy Violation to Patch: Automated Infrastructure as Code Repair via Policy as Code

PIC2 - Master in Computer Science and Engineering
Instituto Superior Técnico, Universidade de Lisboa

Tiago Rodrigues Caldas — 99125*
tiago.rodrigues.caldas@tecnico.ulisboa.pt

Advisor: João Ferreira (IST)

Abstract. As cloud infrastructure grows in complexity, ensuring compliance and security through Infrastructure as Code (IaC) becomes increasingly challenging. Policy as Code (PaC) is an approach to define, manage, and enforce policies through code. Current PaC engines, such as Open Policy Agent (OPA) and HashiCorp Sentinel, allow the detection of policy violations; however, these engines offer no automatic repair support in case of violations. This thesis presents a novel framework that bridges the gap between policy violation detection and automatic repair in IaC environments. By leveraging Large Language Models (LLMs) fine-tuned on a custom *Spec-Bug-Fix* dataset — comprising policies, misconfigured scripts, and their compliant fixes — the system is capable of generating repair patches that satisfy policy constraints. The framework integrates directly with existing PaC engines and supports an iterative validation loop, where each proposed fix is automatically verified for compliance.

Keywords — Policy as Code, Infrastructure as Code, Auto-Repair, Spec-bug-fix dataset

*I declare that this document is an original work of my own authorship and that it fulfills all the requirements of the Code of Conduct and Good Practices of the Universidade de Lisboa (https://nape.tecnico.ulisboa.pt/en/apoio-ao-estudante/documentos-importantes/regulamentos-da-universidade-de-lisboa/).

Contents

1 Introduction

In modern cloud-native environments, Infrastructure as Code (IaC) enables teams to provision and manage infrastructure using code. However, despite the benefits of IaC, managing security, compliance, and operational consistency becomes more difficult for organizations as the infrastructure scales. The more people provisioning the infrastructure, the greater the risk of misconfigurations that compromise security.

Policy as Code (PaC) addresses key limitations of IaC by enabling the definition, management, and enforcement of policies through code. Rather than relying on manual checks, policies are expressed as configuration files or scripts that can be versioned, shared, and seamlessly integrated into CI/CD pipelines. This integration provides developers with immediate feedback whenever a policy violation occurs, enhancing compliance and security throughout the development lifecycle. Policies can be written in different programming languages, depending on the chosen PaC framework. Several solutions exist, including HashiCorp Sentinel [1], which uses its own Sentinel language; Open Policy Agent (OPA) [2], which relies on Rego; Pulumi CrossGuard [3], which allows policies in Python, TypeScript, and JavaScript; AWS CloudFormation Guard [4], which uses a declarative domain-specific language (DSL); and more.

While these PaC frameworks effectively detect policy violations, they all share a common limitation — they do not automatically remediate non-compliant configurations. In recent research, Thiyagarajan et al. (2024) [5] highlight that current AI-driven frameworks mainly focus on detecting configuration drift but lack automated repair mechanisms, requiring developers to manually interpret errors and apply fixes. The need of manual intervention slows down development and introduces the risk of human errors. To address these limitations, we propose an automated repair system leveraging large language models (LLMs) and PaC engines, in order to repair non compliant IaC code. LLMs have shown strong performance in code related tasks [6–10]and natural language understanding, making them well-suited for analyzing misconfigured IaC scripts and generating compliant fixes [11].

1.1 Work Objectives

The main objective of this research is to develop an automated repair system for policy violations in Infrastructure as Code, extending the current capabilities of Policy as Code frameworks. The system will analyze non-compliant configurations scripts and generate appropriate fixes. The repair can be achieved using large language models. To train these models, a dataset will be developed containing common security policies, misconfigured scripts and their corresponding fixes.

Certain limitations remain, as not all violations have a single correct fix. The appropriate repair depends on the specific infrastructure context and the organization's security and compliance requirements, which may be difficult to automate completely. The risk of false positives or incorrect fixes is also crucial, to avoid introducing new errors into production environments.

To address these limitations, a validation pipeline will be implemented to evaluate the effectiveness, security impact, and accuracy of applied fixes.

1.2 Expected Contributions

The expected contributions of this research are as follows:

1. Development of a policy bug-fix dataset. This will be the first structured dataset containing security policies, misconfigured scripts, and their corresponding fixes. It can be used for future research in automated repair for Policy as Code.

2. An automated repair system for Policy as Code. This research introduces a system capable of automatically fixing non-compliant scripts, unlike existing frameworks that only detect policy violations.

3. Evaluation of the system. A validation pipeline will be implemented to evaluate the correctness of applied patches.

4. Integration with existing Policy as Code frameworks. The proposed solution is designed to complement common used frameworks such as Open Policy Agent and HashiCorp Sentinel.

These contributions expect to improve Policy as Code by bridging the gap between detection and repair, improving security, compliance, and developer workflow efficiency.

2 Background

This chapter provides the necessary background for understanding the challenges and opportunities in automated repair for PaC. It begins by introducing IaC and the current security challenges associated. The chapter then addresses key concepts of PaC and explores the current tools and approaches used in this domain. Finally, it examines Large Language Models (LLMs), covering their foundations and architecture through to an overview of different models and their use cases. Each subsection builds upon the previous one to establish the technical foundation required for developing the proposed solution.

2.1 Infrastructure as Code and Security Challenges

Infrastructure as Code transformed the way teams provision and manage infrastructure. Instead of relying on manual processes and settings, developers can provision infrastructure using tools like Puppet [12], Ansible [13], Terraform [14] and more.

As reported in 2023 by Cloud Security Alliance [15], misconfigurations are one of the leading causes to major data breaches. Some incidents have shown the negative consequences, as the Microsoft Power Apps Leak in 2021. It exposed 38 million records of private personal information from major private companies and government entities in the USA, due to default public access settings of tables. Another example was the T-Mobile API hack in 2023. As the API did not have the proper access controls, the attacker was able to retrieve personal data of about 37 million customers.

In the past, organizations would define policies through manual checklists, word documents or ticketing systems. However, as infrastructure scales, managing security, compliance and operational consistency becomes challenging. There is the risk of a developer violating policies and introducing a non-compliant configuration in the infrastructure. To solve this, a new way of defining and managing policies was created.

2.2 Policy as Code

Policy as Code is the idea of writing code in a high-level language to manage and automate policies, as stated by HashiCorp [16]. Developers can now define policies as code and integrate them into CI/CD pipelines, ensuring that any code changes will be checked against the policies written. It can be used in various domains, including security, compliance, infrastructure management and software development.

By codifying security policies, organizations can easily manage and verify who has access to sensitive data and define specific encryption settings. It also enables the automation of compliance checks for different frameworks, such as PCI DSS, HIPAA, CIS Benchmarks and more. For infrastructure management, PaC ensures the definition and enforcement of configurations for cloud resources and network devices, reducing the risk of misconfigurations. Finally, it can also be used to implement coding standards and best practices, providing instant feedback to the developers throughout the development cycle.

PaC and IaC promote a symbiotic relationship inside an organization. While the first one is used to define the safety, compliance, and consistency requirements of the infrastructure, the second one respects them to actually build and configure the entire system. By automating these processes, organizations minimize human errors and accelerate the deployment process.

2.2.1 Policy

A policy is any rule, condition or instruction that regulates IT operations or processes. Figure 1 shows a policy written in Sentinel, a language and framework created by HashiCorp. This policy ensures that

all AWS S3 Buckets in a Terraform plan are configured with a private access control list, which aligns with the best practices for data security.

```
1   import "tfplan/v2" as tfplan
2
3   aws_s3_buckets = filter tfplan.resource_changes as _, rc {
4     rc.type is "aws_s3_bucket" and
5     rc.mode is "managed" and
6     (
7       rc.change.actions contains "create" or
8       rc.change.actions is ["update"])
9   }
10
11  all_aws_s3_buckets_is_private = rule {
12    all aws_s3_buckets as _, aws_s3_bucket {
13      aws_s3_bucket.change.after.acl == "private"
14    }
15  }
16
17  main = rule {
18    all_aws_s3_buckets_is_private
19  }
```

Figure 1: Sentinel policy snippet for AWS S3 Buckets.

It starts by importing the Terraform plan module in line 1, allowing the inspection of the proposed changes. Then it defines the AWS S3 Buckets by using a filter expression that searches for buckets being created or updated through all resources in the Terraform plan, from line 3 to 9. In lines 11–15, it defines the rule that will be used to evaluate the configuration of all filtered buckets. Finally, in lines 17–19, there is the enforcement of the rule which guarantees that the policy is being respected before the deployment of any changes to the infrastructure.

2.2.2 How PaC Work

Policies can be enforced in different ways in CI/CD pipelines. Sentinel, for instance, provides three enforcement levels [17]:

1. Advisory - warns when a policy breaks, but proceeds with the operation.
2. Soft Mandatory - the organization owner or the users with override privileges can proceed with the operation in case of a policy break.
3. Hard Mandatory - does not allow the breaking of any policy during the provision.

Besides automating policy enforcement, Policy as Code also promotes a proactive security posture. The shift-left approach integrates policy checks in earlier stages of the development cycle, reducing the time and cost needed to fix a misconfiguration compared with after deployment. This reduces the risk of vulnerabilities reaching production. Additionally, versioning policies enable the management of policies like code. With version control and collaboration features, organizations improve consistency and traceability across teams. To illustrate the workflow of a PaC integration into a typical DevSecOps pipeline, consider the next example.

1. A security engineer defines a policy that enforces all AWS S3 buckets to have a private ACL. This policy is written in Sentinel and ensures compliance with internal data protection standards (see lines 11–15 in Figure 1).

2. A developer writes Terraform code that provisions an S3 bucket but mistakenly sets the ACL to "public", which violates the defined policy (see line 9 in Figure 2).

3. As part of the CI/CD pipeline, the Terraform plan is automatically passed through Sentinel [1] during the *plan* phase. Sentinel evaluates the proposed infrastructure changes against the defined policy before allowing them to be applied.

4. Sentinel detects the violation in the developer's configuration. As a result, the CI/CD pipeline blocks the deployment, preventing insecure infrastructure from reaching production.

```
1   resource_changes = {
2       "aws_s3_bucket.b": {
3           "address": "aws_s3_bucket.b",
4           "change": {
5               "actions": [
6                   "create"
7               ],
8               "after": {
9                   "acl": "public"
10              },
11          },
12          "mode": "managed",
13          "type": "aws_s3_bucket",
14      },
15  }
```

Figure 2: Terraform plan showing a misconfigured S3 bucket with a public ACL, violating the Sentinel policy.

By incorporating PaC checks into the CI/CD pipeline, developers can catch misconfigurations early, enforce security policies consistently, and ensure that infrastructure changes comply before deployment. However, developers are still responsible for manually understanding and correcting the misconfiguration, which can be time-consuming and error-prone.

This is the kind of problem my thesis aims to address. The goal is to develop a framework that integrates directly into this kind of pipelines. When a PaC engine like Sentinel flags a misconfiguration, the framework will automatically generate a structured prompt containing the IaC code, the relevant policy, violation details, and context. This prompt will then be passed to a Large Language Model fine-tuned with our dataset, which will generate a candidate fix for the misconfigured IaC code. The fix is then re-evaluated automatically by the policy engine — enabling a loop of detection and automated repair, and reducing manual effort.

2.2.3 PaC Tools Overview

As policies can be written in different languages, several tools implement Policy as Code for different use cases. In this section, we review these PaC frameworks, highlighting their capabilities, limitations, and whether they support automated repair.

1. HashiCorp Sentinel [1] – HashiCorp policy framework that uses its own language, Sentinel, and integrates with HashiCorp products such as Consul, Nomad, Terraform, and Vault. It is a language to write policies and a workflow to develop and test policies, independent of the software they will be written to. Sentinel provides immediate feedback if there is a policy violation, but lacks in automatically repair against these violations.

2. Open Policy Agent [2] - An open-source, general-purpose, policy engine that uses a declarative query language called Rego. Rego is used to define policies over structures in microservices, Kubernetes,

CI/CD pipelines, API gateways, and more. While OPA excels at expressive policy definition and enforcement, it does not support automated repair.

3. Pulumi CrossGuard [3] - An open-source tool that allows policies to be written in familiar programming languages like Python, TypeScript and JavaScript. This framework can be applied to Pulumi stacks written in any language. Before, CrossGuard only provided warn/error-based enforcement, requiring developers to manually fix violations. Recently, they introduced a new concept of enforcing policies — remediation policies. Instead of just checking for compliance, they actually fix the problems in place [18]. Although this feature provides automated repair, it still requires the coding of the remediation for each policy violation.

4. AWS CloudFormation Guard [4] - An open-source, general-purpose, command-line interface that checks AWS CloudFormation templates for policy compliance using a simple policy-as-code declarative language. It does not include automated repair capabilities.

5. Azure Policy [19] – Microsoft Azure offers native PaC capabilities via Azure Policy combined with IaC tools like Azure Resource Manager templates (ARM templates). Policies are defined using JSON format or Bicep and can be integrated with CI/CD pipelines. Azure Policy also supports dealing with existing non-compliant resources — remediation tasks [20]- without needing to alter that resource. Policies that use either *deployIfNotExists* or *modify* can have an associated remediation task triggered to correct resources from a non-compliant state and bring them into compliance. Although this feature provides automated repair, it still requires the creation of the specific remediation task.

6. Kyverno [21] – A Kubernetes-native policy engine that allows policies to be written in YAML. Kyverno policies can validate, mutate, generate, and cleanup any Kubernetes resource, including custom resources. Kyverno CLI can be used to apply and test policies off-cluster as part of an IaC and CI/CD pipelines for example. With the ability to mutate non-compliant configurations in place, generate missing resources, and clean up undesired ones, Kyverno can be considered to support automated repair. However, its capabilities are scoped to Kubernetes environments only and do not extend to other IaC platforms.

7. KubeArmor [22] – A runtime enforcement system that focuses on security policies for Kubernetes workloads using Linux Security Modules (LSM), such as AppArmor, SELinux or BPF-LSM. Unlike declarative PaC tools that enforce policies during deployment, KubeArmor monitors and blocks malicious behavior at runtime by providing a set of hardening policies that are based on industry-leading compliance and attack frameworks such as CIS, MITRE, NIST-800-53, and STIGs. Automated repair is not part of its core function, as it operates post-deployment at the OS-level.

8. Kubewarden [23] – A Kubernetes policy engine that allows writing policies in WebAssembly-supported languages like Rust, Go, or AssemblyScript. Policies are implemented as WebAssembly modules and are distributed using regular container registries. Although KubeWarden focuses on secure, high-performance enforcement, it does not currently support automatic repair, emphasizing policy validation over correction.

9. Gatekeeper [24] – A project built on top of Open Policy Agent to enforce Rego policies as Kubernetes admission controls. Gatekeeper enables developers to validate Kubernetes manifest files before deployment, with constraint templates, audit capabilities, and configuration CRDs. Gatekeeper offers an audit mode that highlights violations, but it does not support automated repair. Any repair must be handled externally, either manually or via other automation pipelines.

In general, current policy tools have some limitations. They tend to work exclusively with their vendor's products, limiting IaC to comply with the tool's syntax. The configuration files might need to be uploaded to the cloud, increasing security risks. While a few tools—such as Pulumi CrossGuard, Azure Policy, and Kyverno—do support forms of automated repair, these capabilities are often limited

in scope and demand explicit configuration and custom logic for each policy violation. These limitations increase the chance of human errors and slow down the development cycle. Table 1 summarizes these Policy as Code tools.

Tool	**Policy Language**	**IaC Supported**	**Vendor Lock-in**	**Auto Repair**
Sentinel	Sentinel (HCL)	Terraform, Vault, Consul	HashiCorp	–
OPA	Rego	Terraform, Kubernetes, APIs	–	–
CrossGuard	Python, JS, TS	Pulumi Stacks	Pulumi	Remediation Policies
CFN Guard	CFN DSL	CloudFormation	AWS	–
Azure Policy	JSON / Bicep	ARM, Bicep, Terraform	Microsoft	Remediation Tasks
Kyverno	YAML	Kubernetes only	–	Mutation
KubeArmor	YAML (LSM rules)	Kubernetes Runtime	–	–
Kubewarden	WebAssembly	Kubernetes	–	–
Gatekeeper	Rego	Kubernetes only	–	–

Table 1: Comparison of Policy as Code Tools and Their Repair Capabilities

2.3 LLMs

Large Language Models (LLMs) are artificial intelligence programs capable of recognizing and generating human-like text. They are trained on large sets of data, and use deep learning techniques to analyze unstructured data and learn how language elements like characters, words, and sentences relate. After the initial training, they are fine-tuned or prompt-tuned to perform specific tasks:

1. Fine-tuning: is the process to adjust a pre-trained model on a smaller, task-specific dataset to improve its performance for a particular application. Fine-tuning is about turning general-purpose models into specialized models. For example, Silva et al. (2024) developed RepairLLaMA [10], a fine-tuned language model designed to automatically fix bugs in Java code.

2. Prompt-tuning: involves optimizing how the input to the LLM is structured to get better responses from the model. Instead of changing the model itself, prompt-tuning adjusts how the input is augmented with additional information for specific needs, using a prompt function [11].

The ability of performing any task they are trained to, makes LLMs one of the most versatile emerging technologies. The adoption of LLMs is accelerating across diverse sectors including finance, healthcare, education, software engineering, and entertainment, with documented applications in at least fourteen

industrial domains [25, 26]. Automation is one of LLMs major use cases. They can assist or fully handle tasks related to language, such as customer support, data analysis, and content creation. They can scan large volumes of text data faster than traditional methods, from sources like social media, reviews, and research papers. This enables businesses to better understand market trends and customer feedback. They can assist developers writing code, refactoring, debugging, or generating documentation [10]. This automation not only cuts down on operational costs but also allows human resources to focus on more strategic activities.

In a recent study, Hassanin & Moustafa showed promising results of LLMs in identifying cyber threats and automating security operations [27]. The increasing sophistication of cyber attacks requires proactive technologies for cyber defense, and LLMs offer capabilities to analyze data, detect threats, and automate routine tasks. Their study demonstrated how LLMs can improve security operations by analyzing logs, identifying misconfigurations, and generating remediation instructions at scale.

These capabilities align with the needs of Policy as Code systems, where the challenge is not only to detect non-compliance, but also to suggest or generate the appropriate corrective actions automatically.

2.3.1 How LLMs Work

LLMs use a combination of neural networks, data sets with hundreds of billions of words and phrases, and an architecture known as transformers, in order to understand and generate human-like language. This combination allows computers to draw information from data, create connections and learn patterns about language, making predictions based on probabilities - a process referred as AI inference. LLMs can generate relevant text by estimating the probability of the next word or phrase based on previous context. The downside of predictions is the significant computational and energy resources required for the constant need to calculate and update probabilities.

Neural networks with multiple layers are the foundation of LLMs. These are designed to simulate the way the human brain processes information. In a simple neural network, every node in one layer is connected to every node in the next layer. However, LLMs require a multiple layer architecture to achieve their capabilities. There are two main types of neural architectures that contributed to the evolution of language models:

- Convolutional Neural Networks (CNNs) [28] - designed for image and object recognition, with common use cases including self-driving cars, facial recognition and medical image analysis. They have three different layers:

    1. Convolutional layer - extract features from input data using preconfigured filters. These filters capture local patterns such as lines or simple textures.
    2. Pooling layer - reduce the dimensionality of data by summarizing regions of the input. This enables the network to focus on the most relevant features.
    3. Fully connected layer - connect every neuron in one layer to every neuron in the next. This allows the network to learn complex relationships between features and make high-level predictions.

- Recurrent Neural Networks (RNNs) [29] - designed to recognize patterns in sequential data like text, speech, and time series. They maintain a hidden state that captures information from previous inputs. The output depends on both the current input and the hidden state from the previous time step. RNNs are used when there is the need of immediate predictions based on the most recent inputs, such as speech recognition, machine translation, and time series prediction.

The introduction of the Transformer architecture by Vaswani et al. (2017) marked a significant breakthrough in Natural Language Processing (NLP) [30]. This model was defined as follows:

> *A new simple network architecture based solely on attention mechanisms, dispensing with recurrence and convolutions entirely.*

The Transformer uses an encoder-decoder structure, where both components are built on a stack of identical layers of multi-head self-attention and feed-forward neural networks. Multi-head self-attention is a mechanism that allows models to focus on different parts of the input sequence simultaneously, capturing various relationships and dependencies. Unlike CNNs and RNNs, self-attention computation trades off the order information for parallel computing. To preserve the order of the input sequence, positional encodings are added to the input embeddings. The input text is broken into smaller units called tokens, which are then converted into numerical vectors for further processing.

The encoder processes the input sequence in parallel, using self-attention to capture dependencies across all positions. Each encoder layer applies multi-head self-attention, followed by a feed-forward network, with residual connections and layer normalization.

The decoder generates the output sequence. Each decoder layer includes a masked self-attention mechanism that prevents the model from attending to future tokens, followed by encoder-decoder attention which allows the decoder to focus on relevant parts of the encoded input, and a feed-forward network. The final decoder output is projected through a linear layer and passed through a softmax to produce a probability distribution over the vocabulary.

This parallel processing approach makes Transformers highly efficient and scalable, especially for training on massive datasets — a critical requirement for LLMs. The self-attention mechanisms allowed this architecture to better understand relationships between all positions in the input sequence simultaneously, without the sequential computation constraints of RNNs. Results showed that the Transformer has been used to achieve state-of-the-art results on a variety of NLP tasks, such as translation, summarization, and sentiment analysis [31], establishing the foundation for the current generation of large language models.

2.3.2 LLMs Overview

In recent years, several LLMs have emerged as leading engines in the field of natural language processing, each built upon the Transformer architecture but differentiated by their training objectives, datasets, fine-tuning strategies, and deployment contexts. These models power a wide range of applications, from search engines and chat assistants, to code generation tools.

In general, LLMs fall into three categories [26, 32, 33]:

Encoder-only models. Encoder-only models use just the encoder part of the Transformer architecture. They can attend to all tokens in the input sequence, both forward and backward, in order to understand the full context. They are trained primarily to create contextual representations of the input, which can be used for various tasks such as text classification, information retrieval and code understanding. Examples of encoder-only models are:

- **BERT:** BERT, which stands for Bidirectional Encoder Representations from Transformers, was designed by Google to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. Pre-trained on BookCorpus and English Wikipedia using Masked Language Model (MLM) - the sentences are masked and the model is trained to predict the masked words - and next sentence prediction - by learning the relationships between sentences. BERT is open-source and can be can be finetuned with just one additional output layer to create state-of-the-art models for various downstream tasks, such as text classification, question answering and single sentence tagging tasks. [34]
- **RoBERTa:** An optimized variant of BERT trained longer, with more data and without next sentence prediction [35].
- **CodeBERT:** A Microsoft bimodal pre-trained model for natural language (NL) and programming language (PL), targeted at natural language code search and code documentation generation. [36].
- **ALBERT:** A lighter version of BERT using parameter sharing and factorized embeddings - by decomposing the large vocabulary embedding matrix into two small matrices - to lower memory consumption and increase the training speed of BERT [37].

- **ELECTRA:** A transformer-based model with the same architecture as BERT but introducing a novel pre-training task in which the model uses a replaced token detection objective instead of MLM, improving training efficiency [38].
- **Longformer:** Adapted BERT with a novel sliding window attention mechanism to handle long sequences efficiently [39].

Decoder-only models. Decoder-only models use just the decoder part of the Transformer architecture. They rely on a masked self-attention mechanism - allowing each token to attend to all previous tokens in the sequence - and they are autoregressive - predicting the next token based on prior tokens in a sequence. They are designed for generative tasks like text continuation, chat-based applications, and code generation. Examples of decoder-only models are:

- **GPT (Generative Pre-trained Transformer):** A foundational model designed by OpenAI for text generation, widely used in various applications, such as chatbots and voice assistants, content creation and text generation, language translation, content summarization and conversion, data analysis, coding and healthcare. GPT models use autoregressive language modeling and are trained on vast and diverse data from internet text, books, academic articles, and more. GPT-3, released in 2020, contains 175 billion parameters, making it one of the largest and most powerful language models of its time [40]. GPT-4, released in 2023, expands capabilities further in reasoning, multimodal capabilities such as accepting text and images, and task generalization. The exact size and architecture details of GPT-4 remain proprietary and undisclosed [41].
- **LLaMA:** LLaMA 1 was the first collection of foundation language models developed by Meta, in early 2023, ranging from 7B to 65B parameters. The models were trained on publicly available datasets and the LLaMA-13B outperforms GPT-3 on most benchmarks [42]. The second generation, LLaMA 2, released in July 2023 with sizes of 7B, 13B, and 70B parameters, was trained on 40% more data, has double the context length, and was fine tuned for helpfulness and safety [43]. In 2024, Meta introduced LLaMA 3 models, being the largest a dense Transformer with 405B parameters, that natively support multilinguality, coding, reasoning, and tool usage. LLaMA 3 models approach or surpass GPT-4 in several standard benchmarks and are aligned for safe and helpful responses [44]. Recently, in 2025, Meta released LLaMA 4, LLaMA first models that use a mixture of experts (MoE) architecture - a single token activates only a fraction of the total parameters. MoE architectures are more compute efficient for training and inference and, given a fixed training FLOPs budget, delivers higher quality compared to a dense model [45]. Despite their open-weights nature, LLaMA models require acceptance of Meta's license, restricting commercial and derivative uses.
- **Code LLaMA:** Code LLaMA is a code-specialized variant of Meta's LLaMA series, introduced in 2023 [6]. Unlike the base LLaMA models, Code LLaMA is trained on 500B tokens of code and code-related data, including infilling tasks—the model's ability to generate missing content within a given span of text—using a fill-in-the-middle (FIM) objective. This allows it to generate or repair code by leveraging both preceding and following context. Code LLaMA achieves strong performance on code generation benchmarks such as HumanEval [46] and MBPP [47]. While open-weight, its usage is restricted by Meta's non-commercial research license.
- **Claude:** Based on Constitutional AI principles, Claude is a proprietary chat model, designed by Anthropic, focused on safe and helpful dialog. Specific training data and architecture are not publicly available [48]. Claude 3.5 Sonnet is their strongest vision model yet, outperforming GPT-4o in several evaluations, particularly in reasoning and complex code generation [49].
- **Mixtral:** Open-weight models released by Mistral AI. Mistral 7B is a dense decoder-only model, and Mixtral is a sparse MoE model using 2 active experts per token, promoting inference efficiency. Mixtral demonstrates superior capabilities in mathematics, code generation, and tasks that require

multilingual understanding compared to Llama 2 70B, and outperforms or matches GPT-3.5 across all evaluated benchmarks [50].

- **PaLM / Gemini:** PaLM (Pathways Language Model) is a Google proprietary, decoder-only model trained with the Pathways system and reaching 540B parameters. It demonstrated strong performance on multilingual tasks and source code generation. [51]. Google later released PaLM 2, which became the foundation for many of Google's AI products including Bard (before it transitioned to Gemini). Gemini is Google's newer and more advanced AI model family, launched in late 2023, that exhibit remarkable capabilities across image, audio, video, and text understanding [52].

- **DeepSeek:** DeepSeek-V3 [53], released in December 2024, is a MoE language model with 671 billion parameters (37B active per token), trained on 14.8 trillion tokens in under two months. Despite its scale, it achieved impressive training efficiency with a compute cost of approximately $5.6 million. DeepSeek-V3 surpasses open-source models like LLaMA 3.1–405B, and proprietary models such as GPT-4o and Claude-3.5-Sonnet, particularly in code generation, mathematics, and Chinese language understanding. Following this, DeepSeek-R1 was released in January 2025, targeting advanced reasoning and code-based tasks [9]. Both models are released under an open MIT license, with full weights available for download, and are deployable through platforms like AWS and Azure AI Foundry [54].

- **StarCoder:** StarCoder is an open-source, decoder-only large language model specialized in code generation, developed by the BigCode community and released in 2023 [8]. It is trained on licensed data from GitHub, including from more than 80 programming languages, Git commits, GitHub issues, and Jupyter notebooks. StarCoder supports multiple programming languages and is designed to excel in tasks like code completion, code generation, and code understanding. Its architecture is based on the Transformer decoder with up to 15.5 billion parameters. StarCoder outperform the largest models on the HumanEval [46] benchmark, including PaLM [51] and LLaMA [42].

Encoder-decoder models. Encoder-decoder models - also known as sequence-to-sequence (seq2seq) model - combine the strengths of understanding input and generating output, making them ideal for neural machine translation, text summarization, image captioning and speech recognition. Examples of encoder-decoder models are:

- **T5 (Text-to-Text Transfer Transformer):** T5 is another Google model in a range of sizes from 60M to 11B parameters, released in 2020, that treats every NLP task as a text-to-text problem - both the input and the output are text strings. This model was pretrained on a large corpus called C4 (Colossal Clean Crawled Corpus). [55].

- **FLAN-T5:** A fine-tuned variant of T5 trained on more than 60 NLP datasets expressed via natural language instructions to improve zero-shot performance—the model's ability to solve a task using only its generalized knowledge gained during pretraining, without any specific examples of that task in the prompt or in fine-tuning data. [56].

- **CodeT5 / CodeT5+:** CodeT5 and CodeT5+ are built on the T5 framework but adapted for code, and can be deployed as an AI-powered coding assistant to boost the productivity of software developers. CodeT5+ is an improved encoder-decoder model designed for code intelligence tasks, such as code generation and completion, math programming, and text-to-code retrieval tasks. It extends the original CodeT5 [7] by pretraining on massive code corpora using code spans, partial programs, and complete programs [57]. Both models achieve the best performance among open-source LLMs on challenging benchmarks such as HumanEval [46], outperforming other LLMs such as LLaMA [42], StarCoder [8]. The models are open-source.

In Table 2 we summarize these models, addressing their type, ownership and openness. We define open-source as full source code and model weights publicly released under an open license; open-weights as only model weights released, usually with license restrictions; proprietary as neither code nor weights

Model	Type	Owner	Openness
BERT	Encoder-only	Google	Open-Source
RoBERTa	Encoder-only	Meta	Open-Source
CodeBERT	Encoder-only	Microsoft	Open-Source
ALBERT	Encoder-only	Google	Open-Source
ELECTRA	Encoder-only	Google	Open-Source
Longformer	Encoder-only	AllenAI	Open-Source
GPT-3 / GPT-4	Decoder-only	OpenAI	Proprietary
LLaMA 1–4	Decoder-only	Meta	Open-Weight
Code LLaMA	Decoder-only	Meta	Open-Weight
Claude	Decoder-only	Anthropic	Proprietary
Mixtral	Decoder-only	Mistral AI	Open-Weight
PaLM / Gemini	Decoder-only	Google	Proprietary
DeepSeek-V3/R1	Decoder-only	DeepSeek AI	Open-Weight
StarCoder	Decoder-only	BigCode	Open-Source
T5	Encoder-decoder	Google	Open-Source
FLAN-T5	Encoder-decoder	Google	Open-Source
CodeT5 / T5+	Encoder-decoder	Salesforce	Open-Source

Table 2: Large Language Models categorized by type, ownership, and openness

publicly available. When a company releases weights, they provide developers with the trained model to download and run locally, use for inference (generating text, making predictions), fine-tune on their own data, and study the model's behavior.

2.3.3 Prompt Engineering

Prompt engineering refers to the process of designing and optimizing input prompts to effectively guide the behavior and output of LLMs. A prompt can take various forms — including questions, statements, or instructions — and is the mechanism for interacting with an LLM. Optimizing how the input to the LLM is structured is fundamental to get better responses from the model [58].

Another important concept in this scope is the *temperature* setting. This refers to the randomness and creativeness of the model's responses. Higher values of temperature will result in more diverse and less predictable responses, while lower values will result in more deterministic responses.

There is no single way to write prompts. Different frameworks have been studied, and we will now address some of them:

COSTAR. The COSTAR [59] framework provides a structured approach to prompt creation. COSTAR stands for Context, Objective, Style, Tone, Audience and Response. By considering all these aspects, the model outputs better responses. The framework is explained below:

1. **Context:** Providing background information allows the LLM understanding the scenario.

2. **Objective:** Defining the goal of the tasks ensures the LLM addresses the correct problem.
3. **Style:** Indicating the desired writing style — academic, journalistic, casual — of the response.
4. **Tone:** Specifying emotional tone — optimistic, critical, neutral — of the response.
5. **Audience:** Identifying the intended readers — general public, domain experts, students — adjusts the complexity and vocabulary of the response.
6. **Response:** Providing the expected format — bullet points, JSON, text — of the response.

LangGPT. The LangGPT [60] is a dual-layer structured and extensible framework proved to enhance the performance of LLMs. LangGPT has a systematic structure similar to object-oriented programming languages. It treats prompts like code — organizing them into a core (inherent) layer and an extension layer, composed of well-defined modules and internal elements. As ChatGPT excels at role-playing, they provide a Role template. It consists on four sections:

1. **Profile:** Defining the role's identity, including its description, characteristics, skill sets, and behavioral attributes.
2. **Rules:** Specifying constraints and operational guidelines the role must follow.
3. **Workflow:** Detailing the input format from users and the model's response strategy — First, xxx; Then, xxx; Finally, xxx.
4. **Initialization:** Initializing the role by loading the template's default configuration — "As a/an *Role*, you must follow the *Rules*, you must talk to user in default *Language*,you must greet the user. Then introduce yourself and introduce the *Workflow*."

CRISPE. The CRISPE [61] framework provides a blueprint for creating effective prompts for ChatGPT. It comprises five elements:

1. **Capacity and Role:** Defining the assumed role or expertise of the model.
2. **Insight:** Providing domain-specific background or context required for understanding the task.
3. **Statement:** Specifying the core task, question, or problem the model is asked to address.
4. **Personality:** Setting the tone, voice, or style to shape the model's output.
5. **Experiment** Generating multiple interpretations or creative variations of the response.

Despite their differences in design, the COSTAR, LangGPT, and CRISPE frameworks share a fundamental principle: effective prompt engineering benefits from a structured and systematic approach. By decomposing prompts into clearly defined components — whether it be roles, context, style, or expected outputs — these frameworks enable users to guide LLMs behavior for better results.

3 Related Work

This section explores existing research and systems that leverage machine learning techniques to automate the repair of cloud configurations.

3.1 Bridging the Gap Between Detection and Repair

Recent studies in AI-driven systems for Infrastructure as Code have focused on detecting configuration drift, which is the inconsistency between the declared state in IaC and the actual deployed infrastructure overtime. For example, Thiyagarajan et al. (2024) [5] proposed an AI-based framework that identifies drift in cloud configurations. The system architecture is designed to collect, preprocess, and analyze configuration data to identify deviations from IaC-defined baselines.

The data is collected from three sources: IaC templates, AWS Config, and Azure Config, which provide critical baseline and runtime configurations. After being collected, the data is preprocessed through cleansing, normalization using Pandas and NumPy, and labeling via scikit-learn, and finally classifying configurations as baseline-compliant or drifted. Thiyagarajan et al. chose to implement Random Forest model, also via scikit-learn. The model was trained on historical and synthetic data to detect anomalies and classify configuration states with high accuracy. During runtime, it performed drift detection, flags deviations, generating alerts, and logs results. This architecture reduces manual monitoring efforts and ensures compliance with defined baselines, guaranteeing scalability and adaptability to modern multi-cloud environments.

However, this approach still requires manual intervention to correct misconfigurations.

> *Another critical area for future development is the incorporation of automated remediation mechanism* [5]

The authors highlighted the need for automated remediation mechanisms to streamline the process and reduce downtime. Our work aims to bridge this gap by introducing a machine learning-based system that automatically remediates misconfigurations in real-time.

3.2 InfraFix: Technology-Agnostic Repair of Infrastructure as Code

Automated Program Repair (APR) for IaC is an underexplored topic, largely due to the lack of suitable specifications. InfraFix [62] is the first technology-agnostic APR framework for IaC, supporting multiple languages including Terraform [14], Ansible [13], Puppet [12], and Chef [63]. It is a novel approach that starts by taking information describing the desired system state and inferring the corresponding system state, translating the IaC script into the normalized intermediate GLITCH's [64] representation (IR), and repairing the IR according to the desired system state. It ends with by propagating the changes back to the original IaC script. The repair module integrates an SMT-based approach, extending it to support repairs for: files, packages, services and users in Ansible, Puppet, and Chef; AWS IAM roles, AWS EC2 instances, and AWS S3 buckets in Terraform and Ansible. InfraFix achieved high pass rates in repair scenarios across Ansible (95.8%), Chef (96.5%), Terraform (100.0%), and Tortoise (91.4%), with some failures due to names that could not be statically resolved to strings, and function calls and classes from different files. This work does not leverage machine learning techniques but it shows that automated repair in IaC, though underexplored, has viable solutions, establishing a baseline for my research.

3.3 Deep Learning for Compliance Automation

In a recent study, Thatikonda explored the potential of a compliance automation and deep learning-based remediation techniques to improve security posture in cloud environments. The system proved effectiveness in automating GDPR compliance across 75 European organizations and demonstrated strong capabilities in supporting HIPAA compliance in healthcare settings. Thatikonda proposed:

> *A novel approach to automating regulatory compliance using deep learning techniques, supported by empirical evidence from deployments across major cloud service providers. The proposed system architecture comprises three main components: data collection and preprocessing, model training and inference, and automated remediation, achieving an average end-to-end processing time of 1.8 seconds for compliance validation.* [65]

The first layer of Thatikonda solution - Data Collection Layer - is responsible for implementing continuous monitoring across cloud infrastructure. With this layer, the system is able to process up to 500,000 configurations changes per day, reaching peaks of 1,500 changes per minute. Additionally, it tracks changes across more than 200 configurations parameters, covering an average of 10,000 resources per deployment. With real-time processing latency maintained at 15 milliseconds, this layer ensures fast aggregation of data from diverse sources, aligning with industry best practices for infrastructure real-time monitoring.

Once collected, data flows in the system's second layer - Processing and Analysis Layer - where algorithms extract meaningful patterns and detect potential compliance violations. It supports real-time compliance monitoring by processing up to 1 million events per second with 98.5% accuracy. Additionally, feature extraction process data streams at 600 GB per hour while preserving 99.5% data integrity. Risk assessment and anomaly detection mechanisms evaluate 10,000 metrics simultaneously, ensuring fast and accurate identification of policy violations.

The final stage in the compliance automation pipeline - Automated Remediation Layer - demonstrated a 93.5% success rate in resolving issues autonomously. With a mean time to remediation (MTTR) of just 60 seconds for critical violations, the system is able to process an average of 3,500 remediation actions daily. Additionally, the system's handles up to 1,000 concurrent remediation actions with a 95.5% success rate in non-compliant configuration corrections. These automated actions are supported by a remediation plan generation system, which processes complex compliance scenarios in under 5 seconds and ensures the effectiveness of the remediation through a 99.2% accuracy rate in validation.

This three-layer architecture reduced manual intervention requirements by 85% while improving overall compliance accuracy by 82%, compared to traditional approaches. The results achieved by this study highlights the potential of deep learning for compliance workflows. By effectively reducing the MTTR, the integration of automated remediation proves to be an enhancement for compliance automation.

3.4 Repairing Infrastructure as Code using Large Language Models

Low, Cheh, and Chen (2024) [11] identified the current state-of-the-art approaches to automatic program repair in two categories: heuristic repair and constraint-based repair. Heuristic repair is used for generating potential bug fixes and evaluate them using a certain test suite. In contrast, constraint-based approaches treat the test suite as a collection of conditions that any modified code must meet. To complement the previous two approaches, learning-based repair methods can be used by helping to rank repair candidates in heuristic repair, or be applied on its own for end-to-end repair, by predicting the repaired code for a given piece of code.

In this work, Low, Cheh, and Chen [11] proposed a novel LLM-based repair framework for Infrastructure as Code misconfigurations, written in Terraform language. Their workflow starts with the identification of the misconfigurations in IaC code by security scanners. Each scanner output highlights the location of the misconfiguration in the code. The code is then pre-processed and divided into code blocks using a custom parser, which will be associated with the respective misconfiguration information obtained from the security scanner. This information includes the security scanner name, file path, title and description, line range, severity level, and associated policy ID. Finally, they prompt the LLM to generate the repaired IaC code block.

In many cases, the LLM requires additional details or context from the developer to generate a correct code block. To address this issue, they introduced an *human-in-the-loop* [11] approach. Besides returning the repaired IaC code blocks, they allow the LLM to request for additional context from the developer. The suggested repairs are then applied and the code block is passed again through the security scanners. The code blocks that fail to pass the security scanners are flagged as misconfigurations, and combined with the information requested by the LLM, are passed again to the LLM. The output is the final repaired IaC code.

To validate their workflow, Low, Cheh, and Chen utilized two vulnerable-by-design Terraform codebases - Terragoat [66] and KaiMonkey [67] - which simulate real-world cloud configuration and security issues. The security scanners used were Checkov [68], Terrascan [69], and Trivy [70], which evaluate Terraform resources against defined security policies, flagging violations as misconfigurations. They se-

lected the most recent GPT-3.5 and GPT-4 models as the evaluation targets, configuring both with a temperature of 0 in line with OpenAI's recommendations for factual use cases.

The results show that GPT-4 consistently outperforms GPT-3.5 in repairing Terraform misconfigurations, fixing 18%–34% more issues in a single pass and extending this lead to 57% on a second pass when evaluated with the Checkov scanner. By introducing a human-in-the-loop providing context, the LLMs are able to repair a significant number of misconfigurations. With the additional context, GPT-4 was able to fix 22%–60% more misconfigurations, reaching a fix rate of 87.4% for Checkov-detected issues. An analysis by resource type revealed that Compute, Container, Load Balancer, Network, and Service resources were generally easier to repair, with over 70% of misconfigurations resolved. In contrast, IAM and Security Group resources proved more challenging due to their dependency on broader contextual knowledge of the system and cloud environment.

The authors argue that LLM hallucinations will present an ongoing challenge for the research community. While syntax and semantic errors caused by hallucinations can typically be identified and corrected by language validators, flaws in the LLM's reasoning can lead to superficial "fixes" that, if ignored by developers, could create lasting security vulnerabilities. Despite this, there is clear value in using LLMs for suggesting automated fixes, as they help reduce the manual effort required by developers to identify and repair the misconfigurated code. Consequently, they defend that LLMs are more suitable as an assistant for Infrastructure as Code writing rather than serving as fully automated tools for code repair.

This study by Low, Cheh, and Chen [11] provides a strong foundation for leveraging LLMs in repairing Terraform code using outputs from security scanners [68–70]. However, in contrast to their scanner-driven approach, our work focuses on violations detected by PaC engines [1, 2]. In our framework, policies are treated as specifications rather than just validation tools. The goal is to automatically repair scripts in a way that ensures full compliance with these defined policies, resulting in a more specification-driven and policy-aligned repair process.

4 Solution

To bridge the gap between detection and repair in PaC workflows, we propose a novel framework that leverages PaC engines and LLMs. The system takes as input a policy and an IaC script, the PaC engine automatically identifies violations, and, if there any violations, the LLM generates validated patches iteratively. The key innovation lies in combining static policy enforcement with generative language capabilities to produce context-aware, compliant IaC configurations.

4.1 Spec-Bug-Fix Dataset

To support the training and evaluation of LLMs for IaC repair, we will develop a structured *Spec-Bug-Fix* dataset, labeled according to security violations and their corresponding fix. Each entry consists of:

- **Specification (Policy)**: The policy definition, expressed in Sentinel or Rego.
- **Buggy IaC Script**: The misconfigured IaC script that violates the given policy.
- **Fix (Patch)**: The corrected version of the buggy IaC script that complies with the policy.

The dataset will be constructed by collecting existing open-source IaC misconfigurations and policies examples from tools like Checkov [68], Trivy [70], as well as manual examples where necessary. We acknowledge that manually constructing the "Fix (Patch)" component for each violation can become challenging to scale. As a potential extension, future work could explore reinforcement learning (RL) techniques such as Goal-Reward Policy Optimization (GRPO), where a model learns to iteratively correct IaC scripts by receiving positive reward signals upon policy compliance — when the patched script passes all policy checks. This approach reduces the dependency on labeled patches and leverages RL to generate compliant fixes dynamically. Due to the variable nature of publicly available data and the challenges in sourcing comprehensive labeled fixes, it is not possible at this stage to estimate the final size of the dataset.

The dataset will be structured across two dimensions:

1. **Policy Language Dimension:** Examples are grouped based on the language used to express the policy— *Open Policy Agent (OPA) / Rego* and *HashiCorp Sentinel*. This ensures coverage of the most commonly adopted PaC engines and allows evaluation of LLM performance across syntactic and semantic variations in policy languages.

2. **Violation Category Dimension:** Inspired by real-world security frameworks such as MITRE ATT&CK [71], CIS Benchmarks [72], and supported by rule taxonomies from tools like Checkov [68] and Trivy [70], each example is classified into one of the following categories:

   - *Identity and Access Management (IAM)*
   - *Networking and Firewalls*
   - *Data Protection and Encryption*
   - *Secrets Management*
   - *Logging and Monitoring*
   - *Resource Constraints and Limits*

To scope this initial effort, we will focus primarily on Terraform as the IaC language and Sentinel and Rego as the policy languages. Terraform is one of the most widely adopted declarative IaC language, with strong native integration with both Sentinel and OPA.

Justification of Organization. This dual categorization serves four purposes:

1. **Coverage Diversity:** The dataset captures a wide range of real-world misconfigurations across domains and tools by explicitly changing the policy engine and violation type. By doing this, the model is not overfit to a limited subset of resources or policies.

2. **Availability of examples:** Publicly available policy rule sets and misconfiguration examples are significantly more abundant for Terraform than for alternatives like Pulumi or AWS CloudFormation.

3. **Generalizability for LLM Training:** The inclusion of semantically diverse examples promotes better generalization for LLMs. A model trained on both Sentinel and Rego policies picks up repair techniques that are independent of language.

4. **Fine-grained Evaluation:** Categorization allows quantitative and qualitative evaluation of model performance per violation type and per policy language, making it easier to identify where repair remains challenging.

While our initial experiments will focus primarily on Sentinel-Terraform and OPA-Terraform pairs, future expansions may incorporate other PaC engines and IaC language, depending on availability and community relevance.

Directory Structure. Each entry in the dataset is stored in a structured directory hierarchy:

```
dataset/
|-- opa/
|   |-- iam/
|   |    |-- patch1/
|   |         |-- policy.rego
|   |         |-- buggy.tf
|   |         |-- patch.tf
|   |-- networking/
|       ...
|-- sentinel/
```

```
|   |-- logging/
|   |     |-- patch1/
|   |           |-- policy.sentinel
|   |           |-- buggy.tf
|   |           |-- patch.tf
    ...
```

This schema facilitates reproducibility and scalability-it is easy to add more policies with different languages. It also enables filtering subsets of the dataset for specific evaluation tasks or future studies.

Conclusion. This structured and annotated dataset is the foundation of the project's goals. It ensures a diverse training ground for LLM fine-tuning and supports rigorous evaluation of automated repairs under a wide range of policy and security contexts. For fine-tuning, the dataset will be split into training and validation sets using an 80:20 ratio [73]. This split balances the need for sufficient training data with a representative validation set to monitor and tune model performance during training.

4.2 LLM Selection Criteria

The selection of an appropriate large language model (LLM) is guided by three primary criteria:

1. **Open Access:** The model must be either *open-source* or *open-weights*, as defined earlier. Open-source refers to full public release of both source code and model weights under an open license, while open-weights indicates that only the model weights are publicly available, often under more restrictive terms. In contrast, proprietary models offer neither code nor weights. Models with open access [6, 7, 9, 34–39, 42, 45, 50, 53, 55–57] enable critical capabilities: they allow developers to run inference locally, fine-tune on custom datasets, inspect model behavior, and ensure reproducibility in research. These benefits make openness a foundational requirement for our use case.

2. **Code-Centric Pretraining:** The LLM should have been pretrained on large-scale code corpora. Prior research consistently demonstrates that models exposed to extensive programming data substantially outperform general-purpose LLMs on code-related tasks [6–8, 10, 57]. Accordingly, our focus is restricted to LLMs explicitly designed for programming and software engineering applications.

3. **Infilling Objective and Fine-Tuning Compatibility:** Preference is given to models trained with an infilling objective [6, 74], which has proven particularly effective for tasks like automated code repair. Infilling enables the model to synthesize missing code segments by leveraging both left and right context.

Conclusion. Applying the selection criteria above, we identify a subset of models that best align with our requirements. We select:

- **Code LLaMA** [6], which is open-weight, pretrained on 500B tokens of code and related data, and trained using an infilling (FIM) objective, making it highly suitable for code generation and automated code repair tasks.

- **CodeT5+** [57], which is open-source and designed explicitly for code intelligence tasks. It builds on the encoder-decoder architecture with code-centric pretraining and supports span-level infilling, making it also well-suited for automated code repair tasks.

- **StarCoder** [8], which is open-weight and trained on The Stack dataset with a focus on multilingual code generation. While its infilling capabilities are more limited than Code LLaMA, it still supports basic FIM patterns and performs competitively on code benchmarks.

- **DeepSeek-R1** [9], an open-weight model released under the MIT license. It supports strong reasoning and code-related performance and has shown superior capabilities on LeetCode-style benchmarks and expert level in code competition tasks, although its training objective details are less transparent than Code LLaMA.

These models provide an optimal balance between accessibility for its open access nature, domain-specific pretraining on large code data, and architectural design with infilling and fine-tuning methods. Thus, they serve as strong foundations for developing reliable and reproducible systems in automated code repair and a potential for enhancing current PaC tools.

4.3 Proposed framework

Our proposed framework is designed to automatically repair IaC scripts by leveraging PaC engines with a fine-tuned LLM. Figure 3 illustrates the end-to-end flow of the proposed framework.

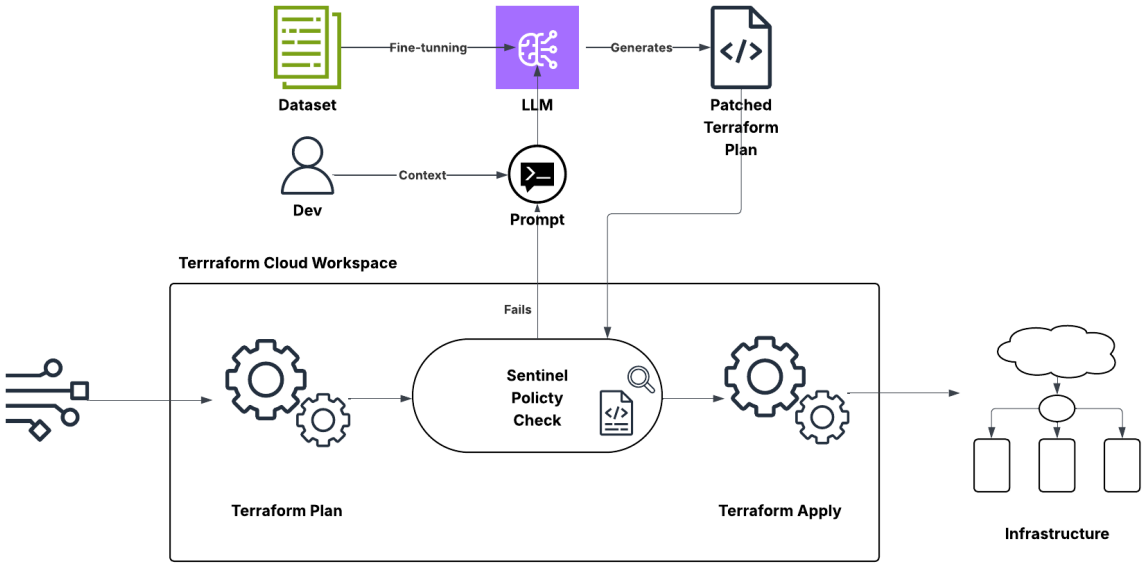

Figure 3: Overview of the proposed PaC-LLM repair framework.

Pipeline Overview. The system accepts two inputs: (1) a policy specification written in a PaC language such as Rego or Sentinel, and (2) an IaC script (e.g., Terraform). The framework performs the following key stages:

1. **Violation Detection:** A PaC engine (Sentinel) analyzes the Terraform plan and flags any policy violations. These are passed forward as structured violation reports.

2. **Contextual Prompt Construction:** Using the violation reports, the corresponding policy definition, and the original script, a structured prompt is dynamically generated. This prompt captures enough context to allow the LLM to synthesize a compliant patch. Different frameworks will be tested to generate better results — COSTAR, LangGPT and CRISPE.

3. **LLM-Driven Repair:** The prompt is fed to a code-centric LLM (e.g., Code LLaMA, CodeT5+, DeepSeek-R1 or StarCoder) fine-tuned with our *Spec-Bug-Fix* dataset, which generates a proposed fix. This output is then optionally passed through an additional verification layer to check compliance with the original policy.

4. **Iterative Validation Loop:** If the patch does not pass validation by the PaC engine, the feedback is recycled into the prompt, and the LLM is invoked again. This loop continues until a compliant patch is produced or a retry limit is reached.

5. ***Human-In-The-Loop* (Optional):** Optionally, the developer can provide additional context, as done before by Low, Cheh, and Chen [11]. This hybrid approach is proved to achieve better results, when additional context is needed for the appropriate repair as shown in the next example.

Example: Human-in-the-Loop Repair with CIDR Block

Policy (Rego): The policy prohibits any inbound access from 0.0.0.0/0— is a CIDR block that means "all IP addresses".

```
deny[msg] {
  input.resource.type == "aws_security_group_rule"
  input.resource.cidr_block == "0.0.0.0/0"
  msg := "Public ingress is not allowed."
}
```

Buggy IaC (Terraform): A Terraform script violates a policy by allowing unrestricted public ingress from `0.0.0.0/0`.

```
resource "aws_security_group_rule" "bad_rule" {
  type            = "ingress"
  from_port       = 22
  to_port         = 22
  protocol        = "tcp"
  cidr_blocks     = ["0.0.0.0/0"]
  security_group_id = aws_security_group.example.id
}
```

Developer Context: SSH access should be restricted to the corporate network at `192.168.100.0/24`.
LLM-Generated Fix:

```
resource "aws_security_group_rule" "restricted_ssh" {
  type            = "ingress"
  from_port       = 22
  to_port         = 22
  protocol        = "tcp"
  cidr_blocks     = ["192.168.100.0/24"]
  security_group_id = aws_security_group.example.id
}
```

Impact: The developer provides context enabling the LLM to generate a fix that is both policy-compliant and operationally correct.

5 Evaluation

The evaluation of the proposed automated repair framework focuses on assessing the correctness and efficiency of the generated IaC patches. To this end, we evaluate in both quantitative and qualitative measures, comparing different model configurations — including base LLMs (zero-shot), prompt-tuned LLMs, and fine-tuned LLMs—on the same benchmark tasks.

5.1 Dataset-Based Evaluation

The foundation for evaluation is the **Spec-Bug-Fix dataset** developed in this thesis (Section 4.1). Each entry consists of a policy, a misconfigured IaC script, and its corresponding correct fix. This dataset is divided into two subsets:

- **Training Set (80%)** — Used during fine-tuning to monitor learning progress and prevent overfitting.
- **Test Set (20% of the full dataset)** — Held out for final evaluation. Only this subset is used to compare model performance.

Test cases are sampled across different policy languages (Sentinel, Rego) and violation categories (e.g., IAM, networking, data protection), allowing fine-grained performance analysis by domain and syntax.

5.2 Experimental Setup

Each IaC input in the test set is passed through different pipelines:

1. **Baseline (Zero-shot)**: Use off-the-shelf LLMs (e.g., Code LLaMA, StarCoder) with no fine-tuning or additional prompt engineering.
2. **Prompt-Tuned**: Use structured prompts (based on frameworks such as COSTAR, LangGPT, and CRISPE) with the same base models.
3. **Fine-Tuned**: Models trained on the Spec-Bug-Fix training data, adapted to the automated IaC repair task.

Each model's output (patch) is evaluated by feeding it back to the corresponding PaC engine (Sentinel or OPA) to verify compliance.

5.3 Evaluation Metrics

We adopt the following quantitative metrics to assess repair performance:

- **Fix Rate (FR)**: The percentage of misconfigured scripts for which the model generates a patch that passes all policy checks on the first attempt. This reflects effectiveness.

$$\text{FR} = \frac{\#\text{ Successful Fixes}}{\#\text{ Total Violations}} \times 100$$

- **Retry Rate (RR)**: The percentage of examples that require more than one repair attempt due to initial failure. This indicates how often the iterative loop (Section 4.3) is needed.
- **Mean Repair Attempts (MRA)**: The average number of iterations required until a compliant patch is found. Lower values are better, indicating higher model efficiency. Also including a timeout threshold for infinite repair attempts.
- **Human-in-the-Loop Efficiency (HLE)**: In evaluations where additional context is provided, we measure how often that context increases the Fix Rate and reduces retries.

All experiments, datasets, prompts, and evaluation scripts will be released as a replication package following the open science policies recommended by SIGSOFT.

6 Work Schedule

A Gantt chart is shown in Figure 4.

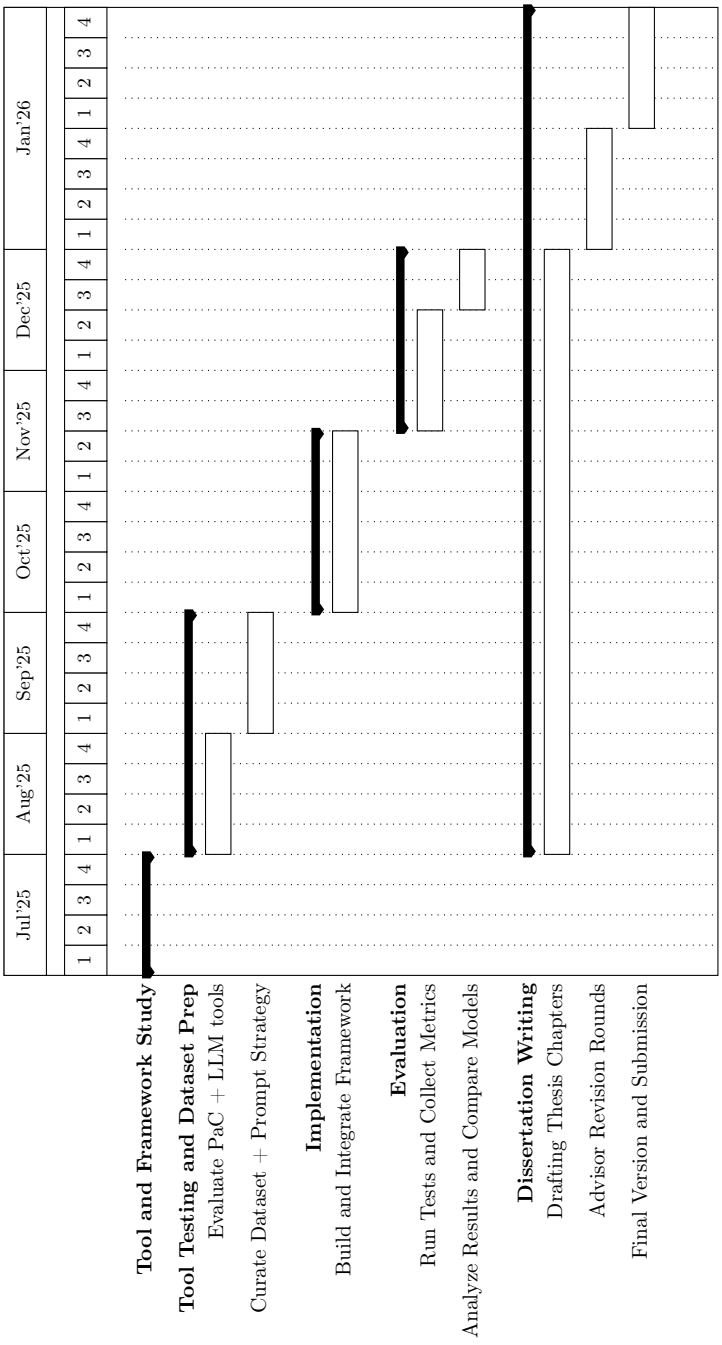

Figure 4: Planned Schedule from July 2025 to January 2026

7 Conclusion

We introduce a framework that bridges the gap between policy violation detection and automated repair in Infrastructure as Code. While existing PaC engines can identify violations, they provide no support for automatic repair — a limitation that remains underexplored in current research. By leveraging Large Language Models fine-tuned on a custom *Spec-Bug-Fix* dataset — policy definitions, violating IaC code, and corresponding fixes — we aim to reduce manual effort and enforce compliance automatically. Our focus is primarily on Terraform as the IaC language and Sentinel and Rego as the policy languages, but future iterations can extend to other languages. However, creating a vast dataset for fine-tuning can become challenging, particularly in terms of scale and coverage. If sufficient examples cannot be obtained, future work may explore reinforcement learning techniques such as Goal-Reward Policy Optimization — allowing models to iteratively correct IaC scripts by receiving positive reward signals for policy compliance, without needing explicit repair examples.

This work is driven by the conviction that PaC automation should not end at detection but extend to policy-driven repairs. As such, the proposed system lays important groundwork for future research and development in automated, policy-compliant IaC workflows.

Bibliography

[1] HashiCorp, "Introduction to sentinel," 2025. [Online]. Available: https://developer.hashicorp.com/sentinel/docs/intro

[2] Styra, "Open policy agent documentation," 2025. [Online]. Available: https://www.openpolicyagent.org/docs/latest/

[3] Pulumi, "Pulumi policy as code," 2025. [Online]. Available: https://www.pulumi.com/docs/iac/using-pulumi/crossguard/#pulumi-policy-as-code

[4] AWS, "What is aws cloudformation guard?" 2025. [Online]. Available: https://docs.aws.amazon.com/cfn-guard/latest/ug/what-is-guard.html

[5] G. Thiyagarajan, V. Bist, and P. Nayak, "Ai-driven configuration drift detection in cloud environments," *International Journal of Communication Networks and Information Security (IJCNIS)*, vol. 16, no. 5, pp. 721–743, 2024. [Online]. Available: https://ssrn.com/abstract=5138379

[6] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code llama: Open foundation models for code," 2024. [Online]. Available: https://arxiv.org/abs/2308.12950

[7] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, "Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *EMNLP*, 2021.

[8] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, T. Wang, O. Dehaene, M. Davaadorj, J. Lamy-Poirier, J. Monteiro, O. Shliazhko, N. Gontier, N. Meade, A. Zebaze, M.-H. Yee, L. K. Umapathi, J. Zhu, B. Lipkin, M. Oblokulov, Z. Wang, R. Murthy, J. Stillerman, S. S. Patel, D. Abulkhanov, M. Zocca, M. Dey, Z. Zhang, N. Fahmy, U. Bhattacharyya, W. Yu, S. Singh, S. Luccioni, P. Villegas, M. Kunakov, F. Zhdanov, M. Romero, T. Lee, N. Timor, J. Ding, C. Schlesinger, H. Schoelkopf, J. Ebert, T. Dao, M. Mishra, A. Gu, J. Robinson, C. J. Anderson, B. Dolan-Gavitt, D. Contractor, S. Reddy, D. Fried, D. Bahdanau, Y. Jernite, C. M. Ferrandis, S. Hughes, T. Wolf, A. Guha, L. von Werra, and H. de Vries, "Starcoder: may the source be with you!" 2023. [Online]. Available: https://arxiv.org/abs/2305.06161

[9] DeepSeek-AI, D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Ding, H. Xin, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Wang, J. Chen, J. Yuan, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, S. Ye, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Zhao, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Xu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan,

Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," 2025. [Online]. Available: https://arxiv.org/abs/2501.12948

[10] A. Silva, S. Fang, and M. Monperrus, "Repairllama: Efficient representations and fine-tuned adapters for program repair," 2024. [Online]. Available: https://arxiv.org/abs/2312.15698

[11] E. Low, C. Cheh, and B. Chen, "Repairing infrastructure-as-code using large language models," in *2024 IEEE Secure Development Conference (SecDev)*, 2024, pp. 20–27.

[12] P. by Perforce, "Puppet documentation," 2024. [Online]. Available: https://www.puppet.com/docs/puppet/6/puppet_overview.html

[13] R. Hat, "Ansible documentation," 2024. [Online]. Available: https://docs.ansible.com/

[14] HashiCorp, "Terraform documentation," 2024. [Online]. Available: https://developer.hashicorp.com/terraform/docs

[15] Cloud Security Alliance, "Cloud security threats to watch out for in 2023: Predictions and mitigation strategies," 2023. [Online]. Available: https://cloudsecurityalliance.org/blog/2023/06/29/cloud-security-threats-to-watch-out-for-in-2023-predictions-and-mitigation-strategies#

[16] HashiCorp, "Sentinel policy as code concepts," 2025. [Online]. Available: https://developer.hashicorp.com/sentinel/docs/concepts/policy-as-code

[17] ——, "Enforce a policy," 2025. [Online]. Available: https://developer.hashicorp.com/terraform/tutorials/cloud-get-started/policy-quickstart

[18] J. Duffy, "Remediation policies: Continuous and automatic compliance," 2023. [Online]. Available: https://www.pulumi.com/blog/remediation-policies/

[19] Microsoft, "What is azure policy?" 2024. [Online]. Available: https://learn.microsoft.com/en-us/azure/governance/policy/overview

[20] ——, "Remediate non-compliant resources with azure policy," 2025. [Online]. Available: https://learn.microsoft.com/en-us/azure/governance/policy/how-to/remediate-resources?tabs=azure-portal

[21] Nirmata, "Kyverno – kubernetes native policy management," 2024. [Online]. Available: https://kyverno.io/

[22] AccuKnox, "Kubearmor: Kubernetes runtime security enforcement," 2024. [Online]. Available: https://kubearmor.io/

[23] SUSE, "Kubewarden - policy-as-code using webassembly," 2024. [Online]. Available: https://www.kubewarden.io/

[24] O. P. Agent, "Gatekeeper - policy controller for kubernetes," 2024. [Online]. Available: https://open-policy-agent.github.io/gatekeeper/website/docs/

[25] J. Jiao, S. Afroogh, K. Chen, D. Atkinson, and A. Dhurandhar, "Generative ai and llms in industry: A text-mining analysis and critical evaluation of guidelines and policy statements across fourteen industrial sectors," 01 2025.

[26] S. Pahune and M. Chandrasekharan, "Several categories of large language models (llms): A short survey," *International Journal for Research in Applied Science and Engineering Technology*, vol. 11, no. 7, p. 615–633, Jul. 2023. [Online]. Available: http://dx.doi.org/10.22214/ijraset.2023.54677

[27] M. Hassanin and N. Moustafa, "A comprehensive overview of large language models (llms) for cyber defences: Opportunities and directions," 2024. [Online]. Available: https://arxiv.org/abs/2405.14487

[28] IBM, "Convolutional neural networks," https://www.ibm.com/think/topics/convolutional-neural-networks, n.d., accessed: 2025-04-17.

[29] ——, "Recurrent neural networks," https://www.ibm.com/think/topics/recurrent-neural-networks, n.d., accessed: 2025-04-17.

[30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://arxiv.org/abs/1706.03762

[31] Dremio, "Transformers in NLP," https://www.dremio.com/wiki/transformers-in-nlp/, 2024, accessed: 2025-05-29.

[32] H. Xu and D. Demner-Fushman, Eds., *Natural Language Processing in Biomedicine: A Practical Guide*. Cham: Springer, 2023.

[33] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," 2023. [Online]. Available: https://arxiv.org/abs/2302.05020

[34] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," 2019. [Online]. Available: https://arxiv.org/abs/1810.04805

[35] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," 2019. [Online]. Available: https://arxiv.org/abs/1907.11692

[36] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "Codebert: A pre-trained model for programming and natural languages," 2020. [Online]. Available: https://arxiv.org/abs/2002.08155

[37] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "Albert: A lite bert for self-supervised learning of language representations," 2020. [Online]. Available: https://arxiv.org/abs/1909.11942

[38] K. Clark, M.-T. Luong, Q. V. Le, and C. D. Manning, "Electra: Pre-training text encoders as discriminators rather than generators," 2020. [Online]. Available: https://arxiv.org/abs/2003.10555

[39] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," 2020. [Online]. Available: https://arxiv.org/abs/2004.05150

[40] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[41] OpenAI, J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, R. Avila, I. Babuschkin, S. Balaji, V. Balcom, P. Baltescu, H. Bao, M. Bavarian, J. Belgum, I. Bello, J. Berdine, G. Bernadett-Shapiro, C. Berner, L. Bogdonoff, O. Boiko, M. Boyd, A.-L. Brakman, G. Brockman, T. Brooks, M. Brundage, K. Button, T. Cai, R. Campbell, A. Cann, B. Carey, C. Carlson, R. Carmichael, B. Chan, C. Chang, F. Chantzis, D. Chen, S. Chen, R. Chen, J. Chen, M. Chen, B. Chess, C. Cho, C. Chu, H. W. Chung, D. Cummings, J. Currier, Y. Dai, C. Decareaux, T. Degry, N. Deutsch, D. Deville, A. Dhar, D. Dohan, S. Dowling, S. Dunning, A. Ecoffet, A. Eleti, T. Eloundou, D. Farhi, L. Fedus, N. Felix, S. P. Fishman, J. Forte, I. Fulford, L. Gao, E. Georges, C. Gibson, V. Goel, T. Gogineni, G. Goh, R. Gontijo-Lopes, J. Gordon, M. Grafstein, S. Gray, R. Greene,

J. Gross, S. S. Gu, Y. Guo, C. Hallacy, J. Han, J. Harris, Y. He, M. Heaton, J. Heidecke, C. Hesse, A. Hickey, W. Hickey, P. Hoeschele, B. Houghton, K. Hsu, S. Hu, X. Hu, J. Huizinga, S. Jain, S. Jain, J. Jang, A. Jiang, R. Jiang, H. Jin, D. Jin, S. Jomoto, B. Jonn, H. Jun, T. Kaftan, Łukasz Kaiser, A. Kamali, I. Kanitscheider, N. S. Keskar, T. Khan, L. Kilpatrick, J. W. Kim, C. Kim, Y. Kim, J. H. Kirchner, J. Kiros, M. Knight, D. Kokotajlo, Łukasz Kondraciuk, A. Kondrich, A. Konstantinidis, K. Kosic, G. Krueger, V. Kuo, M. Lampe, I. Lan, T. Lee, J. Leike, J. Leung, D. Levy, C. M. Li, R. Lim, M. Lin, S. Lin, M. Litwin, T. Lopez, R. Lowe, P. Lue, A. Makanju, K. Malfacini, S. Manning, T. Markov, Y. Markovski, B. Martin, K. Mayer, A. Mayne, B. McGrew, S. M. McKinney, C. McLeavey, P. McMillan, J. McNeil, D. Medina, A. Mehta, J. Menick, L. Metz, A. Mishchenko, P. Mishkin, V. Monaco, E. Morikawa, D. Mossing, T. Mu, M. Murati, O. Murk, D. Mély, A. Nair, R. Nakano, R. Nayak, A. Neelakantan, R. Ngo, H. Noh, L. Ouyang, C. O'Keefe, J. Pachocki, A. Paino, J. Palermo, A. Pantuliano, G. Parascandolo, J. Parish, E. Parparita, A. Passos, M. Pavlov, A. Peng, A. Perelman, F. de Avila Belbute Peres, M. Petrov, H. P. de Oliveira Pinto, Michael, Pokorny, M. Pokrass, V. H. Pong, T. Powell, A. Power, B. Power, E. Proehl, R. Puri, A. Radford, J. Rae, A. Ramesh, C. Raymond, F. Real, K. Rimbach, C. Ross, B. Rotsted, H. Roussez, N. Ryder, M. Saltarelli, T. Sanders, S. Santurkar, G. Sastry, H. Schmidt, D. Schnurr, J. Schulman, D. Selsam, K. Sheppard, T. Sherbakov, J. Shieh, S. Shoker, P. Shyam, S. Sidor, E. Sigler, M. Simens, J. Sitkin, K. Slama, I. Sohl, B. Sokolowsky, Y. Song, N. Staudacher, F. P. Such, N. Summers, I. Sutskever, J. Tang, N. Tezak, M. B. Thompson, P. Tillet, A. Tootoonchian, E. Tseng, P. Tuggle, N. Turley, J. Tworek, J. F. C. Uribe, A. Vallone, A. Vijayvergiya, C. Voss, C. Wainwright, J. J. Wang, A. Wang, B. Wang, J. Ward, J. Wei, C. Weinmann, A. Welihinda, P. Welinder, J. Weng, L. Weng, M. Wiethoff, D. Willner, C. Winter, S. Wolrich, H. Wong, L. Workman, S. Wu, J. Wu, M. Wu, K. Xiao, T. Xu, S. Yoo, K. Yu, Q. Yuan, W. Zaremba, R. Zellers, C. Zhang, M. Zhang, S. Zhao, T. Zheng, J. Zhuang, W. Zhuk, and B. Zoph, "Gpt-4 technical report," 2024. [Online]. Available: https://arxiv.org/abs/2303.08774

[42] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023. [Online]. Available: https://arxiv.org/abs/2302.13971

[43] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open foundation and fine-tuned chat models," 2023. [Online]. Available: https://arxiv.org/abs/2307.09288

[44] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, A. Yang, A. Fan, A. Goyal, A. Hartshorn, A. Yang, A. Mitra, A. Sravankumar, A. Korenev, A. Hinsvark, A. Rao, A. Zhang, A. Rodriguez, A. Gregerson, A. Spataru, B. Roziere, B. Biron, B. Tang, B. Chern, C. Caucheteux, C. Nayak, C. Bi, C. Marra, C. McConnell, C. Keller, C. Touret, C. Wu, C. Wong, C. C. Ferrer, C. Nikolaidis, D. Allonsius, D. Song, D. Pintz, D. Livshits, D. Wyatt, D. Esiobu, D. Choudhary, D. Mahajan, D. Garcia-Olano, D. Perino, D. Hupkes, E. Lakomkin, E. AlBadawy, E. Lobanova, E. Dinan, E. M. Smith, F. Radenovic, F. Guzmán, F. Zhang, G. Synnaeve, G. Lee, G. L. Anderson, G. Thattai, G. Nail, G. Mialon, G. Pang, G. Cucurell, H. Nguyen, H. Korevaar, H. Xu, H. Touvron, I. Zarov, I. A. Ibarra, I. Kloumann, I. Misra, I. Evtimov, J. Zhang, J. Copet, J. Lee, J. Geffert, J. Vranes, J. Park, J. Mahadeokar, J. Shah, J. van der Linde, J. Billock, J. Hong, J. Lee, J. Fu, J. Chi, J. Huang, J. Liu, J. Wang, J. Yu, J. Bitton, J. Spisak, J. Park, J. Rocca, J. Johnstun, J. Saxe, J. Jia, K. V. Alwala, K. Prasad, K. Upasani, K. Plawiak, K. Li, K. Heafield, K. Stone, K. El-Arini,

K. Iyer, K. Malik, K. Chiu, K. Bhalla, K. Lakhotia, L. Rantala-Yeary, L. van der Maaten, L. Chen, L. Tan, L. Jenkins, L. Martin, L. Madaan, L. Malo, L. Blecher, L. Landzaat, L. de Oliveira, M. Muzzi, M. Pasupuleti, M. Singh, M. Paluri, M. Kardas, M. Tsimpoukelli, M. Oldham, M. Rita, M. Pavlova, M. Kambadur, M. Lewis, M. Si, M. K. Singh, M. Hassan, N. Goyal, N. Torabi, N. Bashlykov, N. Bogoychev, N. Chatterji, N. Zhang, O. Duchenne, O. Çelebi, P. Alrassy, P. Zhang, P. Li, P. Vasic, P. Weng, P. Bhargava, P. Dubal, P. Krishnan, P. S. Koura, P. Xu, Q. He, Q. Dong, R. Srinivasan, R. Ganapathy, R. Calderer, R. S. Cabral, R. Stojnic, R. Raileanu, R. Maheswari, R. Girdhar, R. Patel, R. Sauvestre, R. Polidoro, R. Sumbaly, R. Taylor, R. Silva, R. Hou, R. Wang, S. Hosseini, S. Chennabasappa, S. Singh, S. Bell, S. S. Kim, S. Edunov, S. Nie, S. Narang, S. Raparthy, S. Shen, S. Wan, S. Bhosale, S. Zhang, S. Vandenhende, S. Batra, S. Whitman, S. Sootla, S. Collot, S. Gururangan, S. Borodinsky, T. Herman, T. Fowler, T. Sheasha, T. Georgiou, T. Scialom, T. Speckbacher, T. Mihaylov, T. Xiao, U. Karn, V. Goswami, V. Gupta, V. Ramanathan, V. Kerkez, V. Gonguet, V. Do, V. Vogeti, V. Albiero, V. Petrovic, W. Chu, W. Xiong, W. Fu, W. Meers, X. Martinet, X. Wang, X. Wang, X. E. Tan, X. Xia, X. Xie, X. Jia, X. Wang, Y. Goldschlag, Y. Gaur, Y. Babaei, Y. Wen, Y. Song, Y. Zhang, Y. Li, Y. Mao, Z. D. Coudert, Z. Yan, Z. Chen, Z. Papakipos, A. Singh, A. Srivastava, A. Jain, A. Kelsey, A. Shajnfeld, A. Gangidi, A. Victoria, A. Goldstand, A. Menon, A. Sharma, A. Boesenberg, A. Baevski, A. Feinstein, A. Kallet, A. Sangani, A. Teo, A. Yunus, A. Lupu, A. Alvarado, A. Caples, A. Gu, A. Ho, A. Poulton, A. Ryan, A. Ramchandani, A. Dong, A. Franco, A. Goyal, A. Saraf, A. Chowdhury, A. Gabriel, A. Bharambe, A. Eisenman, A. Yazdan, B. James, B. Maurer, B. Leonhardi, B. Huang, B. Loyd, B. D. Paola, B. Paranjape, B. Liu, B. Wu, B. Ni, B. Hancock, B. Wasti, B. Spence, B. Stojkovic, B. Gamido, B. Montalvo, C. Parker, C. Burton, C. Mejia, C. Liu, C. Wang, C. Kim, C. Zhou, C. Hu, C.-H. Chu, C. Cai, C. Tindal, C. Feichtenhofer, C. Gao, D. Civin, D. Beaty, D. Kreymer, D. Li, D. Adkins, D. Xu, D. Testuggine, D. David, D. Parikh, D. Liskovich, D. Foss, D. Wang, D. Le, D. Holland, E. Dowling, E. Jamil, E. Montgomery, E. Presani, E. Hahn, E. Wood, E.-T. Le, E. Brinkman, E. Arcaute, E. Dunbar, E. Smothers, F. Sun, F. Kreuk, F. Tian, F. Kokkinos, F. Ozgenel, F. Caggioni, F. Kanayet, F. Seide, G. M. Florez, G. Schwarz, G. Badeer, G. Swee, G. Halpern, G. Herman, G. Sizov, Guangyi, Zhang, G. Lakshminarayanan, H. Inan, H. Shojanazeri, H. Zou, H. Wang, H. Zha, H. Habeeb, H. Rudolph, H. Suk, H. Aspegren, H. Goldman, H. Zhan, I. Damlaj, I. Molybog, I. Tufanov, I. Leontiadis, I.-E. Veliche, I. Gat, J. Weissman, J. Geboski, J. Kohli, J. Lam, J. Asher, J.-B. Gaya, J. Marcus, J. Tang, J. Chan, J. Zhen, J. Reizenstein, J. Teboul, J. Zhong, J. Jin, J. Yang, J. Cummings, J. Carvill, J. Shepard, J. McPhie, J. Torres, J. Ginsburg, J. Wang, K. Wu, K. H. U, K. Saxena, K. Khandelwal, K. Zand, K. Matosich, K. Veeraraghavan, K. Michelena, K. Li, K. Jagadeesh, K. Huang, K. Chawla, K. Huang, L. Chen, L. Garg, L. A, L. Silva, L. Bell, L. Zhang, L. Guo, L. Yu, L. Moshkovich, L. Wehrstedt, M. Khabsa, M. Avalani, M. Bhatt, M. Mankus, M. Hasson, M. Lennie, M. Reso, M. Groshev, M. Naumov, M. Lathi, M. Keneally, M. Liu, M. L. Seltzer, M. Valko, M. Restrepo, M. Patel, M. Vyatskov, M. Samvelyan, M. Clark, M. Macey, M. Wang, M. J. Hermoso, M. Metanat, M. Rastegari, M. Bansal, N. Santhanam, N. Parks, N. White, N. Bawa, N. Singhal, N. Egebo, N. Usunier, N. Mehta, N. P. Laptev, N. Dong, N. Cheng, O. Chernoguz, O. Hart, O. Salpekar, O. Kalinli, P. Kent, P. Parekh, P. Saab, P. Balaji, P. Rittner, P. Bontrager, P. Roux, P. Dollar, P. Zvyagina, P. Ratanchandani, P. Yuvraj, Q. Liang, R. Alao, R. Rodriguez, R. Ayub, R. Murthy, R. Nayani, R. Mitra, R. Parthasarathy, R. Li, R. Hogan, R. Battey, R. Wang, R. Howes, R. Rinott, S. Mehta, S. Siby, S. J. Bondu, S. Datta, S. Chugh, S. Hunt, S. Dhillon, S. Sidorov, S. Pan, S. Mahajan, S. Verma, S. Yamamoto, S. Ramaswamy, S. Lindsay, S. Lindsay, S. Feng, S. Lin, S. C. Zha, S. Patil, S. Shankar, S. Zhang, S. Zhang, S. Wang, S. Agarwal, S. Sajuyigbe, S. Chintala, S. Max, S. Chen, S. Kehoe, S. Satterfield, S. Govindaprasad, S. Gupta, S. Deng, S. Cho, S. Virk, S. Subramanian, S. Choudhury, S. Goldman, T. Remez, T. Glaser, T. Best, T. Koehler, T. Robinson, T. Li, T. Zhang, T. Matthews, T. Chou, T. Shaked, V. Vontimitta, V. Ajayi, V. Montanez, V. Mohan, V. S. Kumar, V. Mangla, V. Ionescu, V. Poenaru, V. T. Mihailescu, V. Ivanov, W. Li, W. Wang, W. Jiang, W. Bouaziz, W. Constable, X. Tang, X. Wu, X. Wang, X. Wu, X. Gao, Y. Kleinman, Y. Chen, Y. Hu, Y. Jia, Y. Qi, Y. Li, Y. Zhang, Y. Zhang, Y. Adi, Y. Nam, Yu, Wang, Y. Zhao, Y. Hao, Y. Qian, Y. Li, Y. He, Z. Rait,

Z. DeVito, Z. Rosnbrick, Z. Wen, Z. Yang, Z. Zhao, and Z. Ma, "The llama 3 herd of models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[45] Meta AI, "Llama 4 and the next frontier of multimodal intelligence," https://ai.meta.com/blog/llama-4-multimodal-intelligence/, 2025, accessed: 2025-05-28.

[46] Q. Peng, Y. Chai, and X. Li, "Humaneval-xl: A multilingual code generation benchmark for cross-lingual natural language generalization," 2024. [Online]. Available: https://arxiv.org/abs/2402.16694

[47] L. Austin, A. Odena, M. Nye, M. Bosma, J. Michalenko, T. Cai, S. Jain, X. Chen, J. Hoffmann, I. Sutskever *et al.*, "Measuring coding challenge competence with the mbpp dataset," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: https://paperswithcode.com/paper/measuring-coding-challenge-competence-with-the

[48] Anthropic, "Claude: An ai assistant by anthropic," https://www.anthropic.com/claude, 2024, accessed: 2025-05-28.

[49] Anthropic. (2024, June) Claude 3.5 sonnet. Accessed: 2025-05-29. [Online]. Available: https://www.anthropic.com/news/claude-3-5-sonnet

[50] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. R. Lavaud, L. Saulnier, M.-A. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. L. Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mixtral of experts," 2024. [Online]. Available: https://arxiv.org/abs/2401.04088

[51] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann, P. Schuh, K. Shi, S. Tsvyashchenko, J. Maynez, A. Rao, P. Barnes, Y. Tay, N. Shazeer, V. Prabhakaran, E. Reif, N. Du, B. Hutchinson, R. Pope, J. Bradbury, J. Austin, M. Isard, G. Gur-Ari, P. Yin, T. Duke, A. Levskaya, S. Ghemawat, S. Dev, H. Michalewski, X. Garcia, V. Misra, K. Robinson, L. Fedus, D. Zhou, D. Ippolito, D. Luan, H. Lim, B. Zoph, A. Spiridonov, R. Sepassi, D. Dohan, S. Agrawal, M. Omernick, A. M. Dai, T. S. Pillai, M. Pellat, A. Lewkowycz, E. Moreira, R. Child, O. Polozov, K. Lee, Z. Zhou, X. Wang, B. Saeta, M. Diaz, O. Firat, M. Catasta, J. Wei, K. Meier-Hellstern, D. Eck, J. Dean, S. Petrov, and N. Fiedel, "Palm: Scaling language modeling with pathways," 2022. [Online]. Available: https://arxiv.org/abs/2204.02311

[52] G. Team, R. Anil, S. Borgeaud, J.-B. Alayrac, J. Yu, R. Soricut, J. Schalkwyk, A. M. Dai, A. Hauth, K. Millican, D. Silver, M. Johnson, I. Antonoglou, J. Schrittwieser, A. Glaese, J. Chen, E. Pitler, T. Lillicrap, A. Lazaridou, O. Firat, J. Molloy, M. Isard, P. R. Barham, T. Hennigan, B. Lee, F. Viola, M. Reynolds, Y. Xu, R. Doherty, E. Collins, C. Meyer, E. Rutherford, E. Moreira, K. Ayoub, M. Goel, J. Krawczyk, C. Du, E. Chi, H.-T. Cheng, E. Ni, P. Shah, P. Kane, B. Chan, M. Faruqui, A. Severyn, H. Lin, Y. Li, Y. Cheng, A. Ittycheriah, M. Mahdieh, M. Chen, P. Sun, D. Tran, S. Bagri, B. Lakshminarayanan, J. Liu, A. Orban, F. Güra, H. Zhou, X. Song, A. Boffy, H. Ganapathy, S. Zheng, H. Choe, Ágoston Weisz, T. Zhu, Y. Lu, S. Gopal, J. Kahn, M. Kula, J. Pitman, R. Shah, E. Taropa, M. A. Merey, M. Baeuml, Z. Chen, L. E. Shafey, Y. Zhang, O. Sercinoglu, G. Tucker, E. Piqueras, M. Krikun, I. Barr, N. Savinov, I. Danihelka, B. Roelofs, A. White, A. Andreassen, T. von Glehn, L. Yagati, M. Kazemi, L. Gonzalez, M. Khalman, J. Sygnowski, A. Frechette, C. Smith, L. Culp, L. Proleev, Y. Luan, X. Chen, J. Lottes, N. Schucher, F. Lebron, A. Rrustemi, N. Clay, P. Crone, T. Kocisky, J. Zhao, B. Perz, D. Yu, H. Howard, A. Bloniarz, J. W. Rae, H. Lu, L. Sifre, M. Maggioni, F. Alcober, D. Garrette, M. Barnes, S. Thakoor, J. Austin, G. Barth-Maron, W. Wong, R. Joshi, R. Chaabouni, D. Fatiha, A. Ahuja, G. S. Tomar, E. Senter, M. Chadwick, I. Kornakov, N. Attaluri, I. Iturrate, R. Liu, Y. Li, S. Cogan, J. Chen, C. Jia, C. Gu, Q. Zhang, J. Grimstad, A. J. Hartman, X. Garcia, T. S. Pillai, J. Devlin, M. Laskin, D. de Las Casas, D. Valter, C. Tao, L. Blanco, A. P. Badia, D. Reitter,

M. Chen, J. Brennan, C. Rivera, S. Brin, S. Iqbal, G. Surita, J. Labanowski, A. Rao, S. Winkler, E. Parisotto, Y. Gu, K. Olszewska, R. Addanki, A. Miech, A. Louis, D. Teplyashin, G. Brown, E. Catt, J. Balaguer, J. Xiang, P. Wang, Z. Ashwood, A. Briukhov, A. Webson, S. Ganapathy, S. Sanghavi, A. Kannan, M.-W. Chang, A. Stjerngren, J. Djolonga, Y. Sun, A. Bapna, M. Aitchison, P. Pejman, H. Michalewski, T. Yu, C. Wang, J. Love, J. Ahn, D. Bloxwich, K. Han, P. Humphreys, T. Sellam, J. Bradbury, V. Godbole, S. Samangooei, B. Damoc, A. Kaskasoli, S. M. R. Arnold, V. Vasudevan, S. Agrawal, J. Riesa, D. Lepikhin, R. Tanburn, S. Srinivasan, H. Lim, S. Hodkinson, P. Shyam, J. Ferret, S. Hand, A. Garg, T. L. Paine, J. Li, Y. Li, M. Giang, A. Neitz, Z. Abbas, S. York, M. Reid, E. Cole, A. Chowdhery, D. Das, D. Rogozińska, V. Nikolaev, P. Sprechmann, Z. Nado, L. Zilka, F. Prost, L. He, M. Monteiro, G. Mishra, C. Welty, J. Newlan, D. Jia, M. Allamanis, C. H. Hu, R. de Liedekerke, J. Gilmer, C. Saroufim, S. Rijhwani, S. Hou, D. Shrivastava, A. Baddepudi, A. Goldin, A. Ozturel, A. Cassirer, Y. Xu, D. Sohn, D. Sachan, R. K. Amplayo, C. Swanson, D. Petrova, S. Narayan, A. Guez, S. Brahma, J. Landon, M. Patel, R. Zhao, K. Villela, L. Wang, W. Jia, M. Rahtz, M. Giménez, L. Yeung, J. Keeling, P. Georgiev, D. Mincu, B. Wu, S. Haykal, R. Saputro, K. Vodrahalli, J. Qin, Z. Cankara, A. Sharma, N. Fernando, W. Hawkins, B. Neyshabur, S. Kim, A. Hutter, P. Agrawal, A. Castro-Ros, G. van den Driessche, T. Wang, F. Yang, S. yiin Chang, P. Komarek, R. McIlroy, M. Lučić, G. Zhang, W. Farhan, M. Sharman, P. Natsev, P. Michel, Y. Bansal, S. Qiao, K. Cao, S. Shakeri, C. Butterfield, J. Chung, P. K. Rubenstein, S. Agrawal, A. Mensch, K. Soparkar, K. Lenc, T. Chung, A. Pope, L. Maggiore, J. Kay, P. Jhakra, S. Wang, J. Maynez, M. Phuong, T. Tobin, A. Tacchetti, M. Trebacz, K. Robinson, Y. Katariya, S. Riedel, P. Bailey, K. Xiao, N. Ghelani, L. Aroyo, A. Slone, N. Houlsby, X. Xiong, Z. Yang, E. Gribovskaya, J. Adler, M. Wirth, L. Lee, M. Li, T. Kagohara, J. Pavagadhi, S. Bridgers, A. Bortsova, S. Ghemawat, Z. Ahmed, T. Liu, R. Powell, V. Bolina, M. Iinuma, P. Zablotskaia, J. Besley, D.-W. Chung, T. Dozat, R. Comanescu, X. Si, J. Greer, G. Su, M. Polacek, R. L. Kaufman, S. Tokumine, H. Hu, E. Buchatskaya, Y. Miao, M. Elhawaty, A. Siddhant, N. Tomasev, J. Xing, C. Greer, H. Miller, S. Ashraf, A. Roy, Z. Zhang, A. Ma, A. Filos, M. Besta, R. Blevins, T. Klimenko, C.-K. Yeh, S. Changpinyo, J. Mu, O. Chang, M. Pajarskas, C. Muir, V. Cohen, C. L. Lan, K. Haridasan, A. Marathe, S. Hansen, S. Douglas, R. Samuel, M. Wang, S. Austin, C. Lan, J. Jiang, J. Chiu, J. A. Lorenzo, L. L. Sjösund, S. Cevey, Z. Gleicher, T. Avrahami, A. Boral, H. Srinivasan, V. Selo, R. May, K. Aisopos, L. Hussenot, L. B. Soares, K. Baumli, M. B. Chang, A. Recasens, B. Caine, A. Pritzel, F. Pavetic, F. Pardo, A. Gergely, J. Frye, V. Ramasesh, D. Horgan, K. Badola, N. Kassner, S. Roy, E. Dyer, V. C. Campos, A. Tomala, Y. Tang, D. E. Badawy, E. White, B. Mustafa, O. Lang, A. Jindal, S. Vikram, Z. Gong, S. Caelles, R. Hemsley, G. Thornton, F. Feng, W. Stokowiec, C. Zheng, P. Thacker, Çağlar Ünlü, Z. Zhang, M. Saleh, J. Svensson, M. Bileschi, P. Patil, A. Anand, R. Ring, K. Tsihlas, A. Vezer, M. Selvi, T. Shevlane, M. Rodriguez, T. Kwiatkowski, S. Daruki, K. Rong, A. Dafoe, N. FitzGerald, K. Gu-Lemberg, M. Khan, L. A. Hendricks, M. Pellat, V. Feinberg, J. Cobon-Kerr, T. Sainath, M. Rauh, S. H. Hashemi, R. Ives, Y. Hasson, E. Noland, Y. Cao, N. Byrd, L. Hou, Q. Wang, T. Sottiaux, M. Paganini, J.-B. Lespiau, A. Moufarek, S. Hassan, K. Shivakumar, J. van Amersfoort, A. Mandhane, P. Joshi, A. Goyal, M. Tung, A. Brock, H. Sheahan, V. Misra, C. Li, N. Rakićević, M. Dehghani, F. Liu, S. Mittal, J. Oh, S. Noury, E. Sezener, F. Huot, M. Lamm, N. D. Cao, C. Chen, S. Mudgal, R. Stella, K. Brooks, G. Vasudevan, C. Liu, M. Chain, N. Melinkeri, A. Cohen, V. Wang, K. Seymore, S. Zubkov, R. Goel, S. Yue, S. Krishnakumaran, B. Albert, N. Hurley, M. Sano, A. Mohananey, J. Joughin, E. Filonov, T. Kępa, Y. Eldawy, J. Lim, R. Rishi, S. Badiezadegan, T. Bos, J. Chang, S. Jain, S. G. S. Padmanabhan, S. Puttagunta, K. Krishna, L. Baker, N. Kalb, V. Bedapudi, A. Kurzrok, S. Lei, A. Yu, O. Litvin, X. Zhou, Z. Wu, S. Sobell, A. Siciliano, A. Papir, R. Neale, J. Bragagnolo, T. Toor, T. Chen, V. Anklin, F. Wang, R. Feng, M. Gholami, K. Ling, L. Liu, J. Walter, H. Moghaddam, A. Kishore, J. Adamek, T. Mercado, J. Mallinson, S. Wandekar, S. Cagle, E. Ofek, G. Garrido, C. Lombriser, M. Mukha, B. Sun, H. R. Mohammad, J. Matak, Y. Qian, V. Peswani, P. Janus, Q. Yuan, L. Schelin, O. David, A. Garg, Y. He, O. Duzhyi, A. Älgmyr, T. Lottaz, Q. Li, V. Yadav, L. Xu, A. Chinien, R. Shivanna, A. Chuklin, J. Li, C. Spadine, T. Wolfe, K. Mohamed, S. Das, Z. Dai, K. He, D. von Dincklage, S. Upadhyay, A. Maurya, L. Chi, S. Krause, K. Salama, P. G. Rabinovitch,

P. K. R. M, A. Selvan, M. Dektiarev, G. Ghiasi, E. Guven, H. Gupta, B. Liu, D. Sharma, I. H. Shtacher, S. Paul, O. Akerlund, F.-X. Aubet, T. Huang, C. Zhu, E. Zhu, E. Teixeira, M. Fritze, F. Bertolini, L.-E. Marinescu, M. Bölle, D. Paulus, K. Gupta, T. Latkar, M. Chang, J. Sanders, R. Wilson, X. Wu, Y.-X. Tan, L. N. Thiet, T. Doshi, S. Lall, S. Mishra, W. Chen, T. Luong, S. Benjamin, J. Lee, E. Andrejczuk, D. Rabiej, V. Ranjan, K. Styrc, P. Yin, J. Simon, M. R. Harriott, M. Bansal, A. Robsky, G. Bacon, D. Greene, D. Mirylenka, C. Zhou, O. Sarvana, A. Goyal, S. Andermatt, P. Siegler, B. Horn, A. Israel, F. Pongetti, C.-W. L. Chen, M. Selvatici, P. Silva, K. Wang, J. Tolins, K. Guu, R. Yogev, X. Cai, A. Agostini, M. Shah, H. Nguyen, N. O. Donnaile, S. Pereira, L. Friso, A. Stambler, A. Kurzrok, C. Kuang, Y. Romanikhin, M. Geller, Z. Yan, K. Jang, C.-C. Lee, W. Fica, E. Malmi, Q. Tan, D. Banica, D. Balle, R. Pham, Y. Huang, D. Avram, H. Shi, J. Singh, C. Hidey, N. Ahuja, P. Saxena, D. Dooley, S. P. Potharaju, E. O'Neill, A. Gokulchandran, R. Foley, K. Zhao, M. Dusenberry, Y. Liu, P. Mehta, R. Kotikalapudi, C. Safranek-Shrader, A. Goodman, J. Kessinger, E. Globen, P. Kolhar, C. Gorgolewski, A. Ibrahim, Y. Song, A. Eichenbaum, T. Brovelli, S. Potluri, P. Lahoti, C. Baetu, A. Ghorbani, C. Chen, A. Crawford, S. Pal, M. Sridhar, P. Gurita, A. Mujika, I. Petrovski, P.-L. Cedoz, C. Li, S. Chen, N. D. Santo, S. Goyal, J. Punjabi, K. Kappaganthu, C. Kwak, P. LV, S. Velury, H. Choudhury, J. Hall, P. Shah, R. Figueira, M. Thomas, M. Lu, T. Zhou, C. Kumar, T. Jurdi, S. Chikkerur, Y. Ma, A. Yu, S. Kwak, V. Ähdel, S. Rajayogam, T. Choma, F. Liu, A. Barua, C. Ji, J. H. Park, V. Hellendoorn, A. Bailey, T. Bilal, H. Zhou, M. Khatir, C. Sutton, W. Rzadkowski, F. Macintosh, R. Vij, K. Shagin, P. Medina, C. Liang, J. Zhou, P. Shah, Y. Bi, A. Dankovics, S. Banga, S. Lehmann, M. Bredesen, Z. Lin, J. E. Hoffmann, J. Lai, R. Chung, K. Yang, N. Balani, A. Bražinskas, A. Sozanschi, M. Hayes, H. F. Alcalde, P. Makarov, W. Chen, A. Stella, L. Snijders, M. Mandl, A. Kärrman, P. Nowak, X. Wu, A. Dyck, K. Vaidyanathan, R. R, J. Mallet, M. Rudominer, E. Johnston, S. Mittal, A. Udathu, J. Christensen, V. Verma, Z. Irving, A. Santucci, G. Elsayed, E. Davoodi, M. Georgiev, I. Tenney, N. Hua, G. Cideron, E. Leurent, M. Alnahlawi, I. Georgescu, N. Wei, I. Zheng, D. Scandinaro, H. Jiang, J. Snoek, M. Sundararajan, X. Wang, Z. Ontiveros, I. Karo, J. Cole, V. Rajashekhar, L. Tumeh, E. Ben-David, R. Jain, J. Uesato, R. Datta, O. Bunyan, S. Wu, J. Zhang, P. Stanczyk, Y. Zhang, D. Steiner, S. Naskar, M. Azzam, M. Johnson, A. Paszke, C.-C. Chiu, J. S. Elias, A. Mohiuddin, F. Muhammad, J. Miao, A. Lee, N. Vieillard, J. Park, J. Zhang, J. Stanway, D. Garmon, A. Karmarkar, Z. Dong, J. Lee, A. Kumar, L. Zhou, J. Evens, W. Isaac, G. Irving, E. Loper, M. Fink, I. Arkatkar, N. Chen, I. Shafran, I. Petrychenko, Z. Chen, J. Jia, A. Levskaya, Z. Zhu, P. Grabowski, Y. Mao, A. Magni, K. Yao, J. Snaider, N. Casagrande, E. Palmer, P. Suganthan, A. Castaño, I. Giannoumis, W. Kim, M. Rybiński, A. Sreevatsa, J. Prendki, D. Soergel, A. Goedeckemeyer, W. Gierke, M. Jafari, M. Gaba, J. Wiesner, D. G. Wright, Y. Wei, H. Vashisht, Y. Kulizhskaya, J. Hoover, M. Le, L. Li, C. Iwuanyanwu, L. Liu, K. Ramirez, A. Khorlin, A. Cui, T. LIN, M. Wu, R. Aguilar, K. Pallo, A. Chakladar, G. Perng, E. A. Abellan, M. Zhang, I. Dasgupta, N. Kushman, I. Penchev, A. Repina, X. Wu, T. van der Weide, P. Ponnapalli, C. Kaplan, J. Simsa, S. Li, O. Dousse, F. Yang, J. Piper, N. Ie, R. Pasumarthi, N. Lintz, A. Vijayakumar, D. Andor, P. Valenzuela, M. Lui, C. Paduraru, D. Peng, K. Lee, S. Zhang, S. Greene, D. D. Nguyen, P. Kurylowicz, C. Hardin, L. Dixon, L. Janzer, K. Choo, Z. Feng, B. Zhang, A. Singhal, D. Du, D. McKinnon, N. Antropova, T. Bolukbasi, O. Keller, D. Reid, D. Finchelstein, M. A. Raad, R. Crocker, P. Hawkins, R. Dadashi, C. Gaffney, K. Franko, A. Bulanova, R. Leblond, S. Chung, H. Askham, L. C. Cobo, K. Xu, F. Fischer, J. Xu, C. Sorokin, C. Alberti, C.-C. Lin, C. Evans, A. Dimitriev, H. Forbes, D. Banarse, Z. Tung, M. Omernick, C. Bishop, R. Sterneck, R. Jain, J. Xia, E. Amid, F. Piccinno, X. Wang, P. Banzal, D. J. Mankowitz, A. Polozov, V. Krakovna, S. Brown, M. Bateni, D. Duan, V. Firoiu, M. Thotakuri, T. Natan, M. Geist, S. tan Girgin, H. Li, J. Ye, O. Roval, R. Tojo, M. Kwong, J. Lee-Thorp, C. Yew, D. Sinopalnikov, S. Ramos, J. Mellor, A. Sharma, K. Wu, D. Miller, N. Sonnerat, D. Vnukov, R. Greig, J. Beattie, E. Caveness, L. Bai, J. Eisenschlos, A. Korchemniy, T. Tsai, M. Jasarevic, W. Kong, P. Dao, Z. Zheng, F. Liu, F. Yang, R. Zhu, T. H. Teh, J. Sanmiya, E. Gladchenko, N. Trdin, D. Toyama, E. Rosen, S. Tavakkol, L. Xue, C. Elkind, O. Woodman, J. Carpenter, G. Papamakarios, R. Kemp, S. Kafle, T. Grunina, R. Sinha, A. Talbert, D. Wu, D. Owusu-Afriyie, C. Du, C. Thornton, J. Pont-Tuset, P. Narayana, J. Li,

S. Fatehi, J. Wieting, O. Ajmeri, B. Uria, Y. Ko, L. Knight, A. Héliou, N. Niu, S. Gu, C. Pang, Y. Li, N. Levine, A. Stolovich, R. Santamaria-Fernandez, S. Goenka, W. Yustalim, R. Strudel, A. Elqursh, C. Deck, H. Lee, Z. Li, K. Levin, R. Hoffmann, D. Holtmann-Rice, O. Bachem, S. Arora, C. Koh, S. H. Yeganeh, S. Põder, M. Tariq, Y. Sun, L. Ionita, M. Seyedhosseini, P. Tafti, Z. Liu, A. Gulati, J. Liu, X. Ye, B. Chrzaszcz, L. Wang, N. Sethi, T. Li, B. Brown, S. Singh, W. Fan, A. Parisi, J. Stanton, V. Koverkathu, C. A. Choquette-Choo, Y. Li, T. Lu, A. Ittycheriah, P. Shroff, M. Varadarajan, S. Bahargam, R. Willoughby, D. Gaddy, G. Desjardins, M. Cornero, B. Robenek, B. Mittal, B. Albrecht, A. Shenoy, F. Moiseev, H. Jacobsson, A. Ghaffarkhah, M. Rivière, A. Walton, C. Crepy, A. Parrish, Z. Zhou, C. Farabet, C. Radebaugh, P. Srinivasan, C. van der Salm, A. Fidjeland, S. Scellato, E. Latorre-Chimoto, H. Klimczak-Plucińska, D. Bridson, D. de Cesare, T. Hudson, P. Mendolicchio, L. Walker, A. Morris, M. Mauger, A. Guseynov, A. Reid, S. Odoom, L. Loher, V. Cotruta, M. Yenugula, D. Grewe, A. Petrushkina, T. Duerig, A. Sanchez, S. Yadlowsky, A. Shen, A. Globerson, L. Webb, S. Dua, D. Li, S. Bhupatiraju, D. Hurt, H. Qureshi, A. Agarwal, T. Shani, M. Eyal, A. Khare, S. R. Belle, L. Wang, C. Tekur, M. S. Kale, J. Wei, R. Sang, B. Saeta, T. Liechty, Y. Sun, Y. Zhao, S. Lee, P. Nayak, D. Fritz, M. R. Vuyyuru, J. Aslanides, N. Vyas, M. Wicke, X. Ma, E. Eltyshev, N. Martin, H. Cate, J. Manyika, K. Amiri, Y. Kim, X. Xiong, K. Kang, F. Luisier, N. Tripuraneni, D. Madras, M. Guo, A. Waters, O. Wang, J. Ainslie, J. Baldridge, H. Zhang, G. Pruthi, J. Bauer, F. Yang, R. Mansour, J. Gelman, Y. Xu, G. Polovets, J. Liu, H. Cai, W. Chen, X. Sheng, E. Xue, S. Ozair, C. Angermueller, X. Li, A. Sinha, W. Wang, J. Wiesinger, E. Koukoumidis, Y. Tian, A. Iyer, M. Gurumurthy, M. Goldenson, P. Shah, M. Blake, H. Yu, A. Urbanowicz, J. Palomaki, C. Fernando, K. Durden, H. Mehta, N. Momchev, E. Rahimtoroghi, M. Georgaki, A. Raul, S. Ruder, M. Redshaw, J. Lee, D. Zhou, K. Jalan, D. Li, B. Hechtman, P. Schuh, M. Nasr, K. Milan, V. Mikulik, J. Franco, T. Green, N. Nguyen, J. Kelley, A. Mahendru, A. Hu, J. Howland, B. Vargas, J. Hui, K. Bansal, V. Rao, R. Ghiya, E. Wang, K. Ye, J. M. Sarr, M. M. Preston, M. Elish, S. Li, A. Kaku, J. Gupta, I. Pasupat, D.-C. Juan, M. Someswar, T. M., X. Chen, A. Amini, A. Fabrikant, E. Chu, X. Dong, A. Muthal, S. Buthpitiya, S. Jauhari, N. Hua, U. Khandelwal, A. Hitron, J. Ren, L. Rinaldi, S. Drath, A. Dabush, N.-J. Jiang, H. Godhia, U. Sachs, A. Chen, Y. Fan, H. Taitelbaum, H. Noga, Z. Dai, J. Wang, C. Liang, J. Hamer, C.-S. Ferng, C. Elkind, A. Atias, P. Lee, V. Listík, M. Carlen, J. van de Kerkhof, M. Pikus, K. Zaher, P. Müller, S. Zykova, R. Stefanec, V. Gatsko, C. Hirnschall, A. Sethi, X. F. Xu, C. Ahuja, B. Tsai, A. Stefanoiu, B. Feng, K. Dhandhania, M. Katyal, A. Gupta, A. Parulekar, D. Pitta, J. Zhao, V. Bhatia, Y. Bhavnani, O. Alhadlaq, X. Li, P. Danenberg, D. Tu, A. Pine, V. Filippova, A. Ghosh, B. Limonchik, B. Urala, C. K. Lanka, D. Clive, Y. Sun, E. Li, H. Wu, K. Hongtongsak, I. Li, K. Thakkar, K. Omarov, K. Majmundar, M. Alverson, M. Kucharski, M. Patel, M. Jain, M. Zabelin, P. Pelagatti, R. Kohli, S. Kumar, J. Kim, S. Sankar, V. Shah, L. Ramachandruni, X. Zeng, B. Bariach, L. Weidinger, T. Vu, A. Andreev, A. He, K. Hui, S. Kashem, A. Subramanya, S. Hsiao, D. Hassabis, K. Kavukcuoglu, A. Sadovsky, Q. Le, T. Strohman, Y. Wu, S. Petrov, J. Dean, and O. Vinyals, "Gemini: A family of highly capable multimodal models," 2025. [Online]. Available: https://arxiv.org/abs/2312.11805

[53] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin, X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang,

Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," 2025. [Online]. Available: https://arxiv.org/abs/2412.19437

[54] G. Engineering. (2024) Evaluating llms for infrastructure as code. Accessed: 2025-05-29. [Online]. Available: https://medium.com/gft-engineering/evaluating-llms-for-infrastructure-as-code-9f8b9ac4ca33

[55] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," 2023. [Online]. Available: https://arxiv.org/abs/1910.10683

[56] J. Wei, M. Bosma, V. Y. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," 2022. [Online]. Available: https://arxiv.org/abs/2109.01652

[57] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, "Codet5+: Open code large language models for code understanding and generation," 2023. [Online]. Available: https://arxiv.org/abs/2305.07922

[58] GovTech Data Science & AI Division, "Prompt engineering playbook (beta v3)," https://www.developer.tech.gov.sg/products/collections/data-science-and-artificial-intelligence/playbooks/prompt-engineering-playbook-beta-v3.pdf, Government Technology Agency of Singapore, Aug. 2023, last updated 30 August 2023.

[59] F. Zentennial, "Unlocking the power of costar prompt engineering: A guide and example on converting goals into system of actionable items," *Medium*, Jan. 2024.

[60] M. Wang, Y. Liu, X. Liang, S. Li, Y. Huang, X. Zhang, S. Shen, C. Guan, D. Wang, S. Feng, H. Zhang, Y. Zhang, M. Zheng, and C. Zhang, "Langgpt: Rethinking structured reusable prompt design framework for llms from the programming language," 2024. [Online]. Available: https://arxiv.org/abs/2402.16929

[61] D. Dinkevych, "Crispe — chatgpt prompt engineering framework," *Medium*, May 2023.

[62] N. Saavedra, J. F. Ferreira, and A. Mendes, "Infrafix: Technology-agnostic repair of infrastructure as code," 2025. [Online]. Available: https://arxiv.org/abs/2503.17220

[63] Chef Software, Inc., "Chef: Infrastructure as code," 2024. [Online]. Available: https://docs.chef.io

[64] N. Saavedra, J. Gonçalves, M. Henriques, J. F. Ferreira, and A. Mendes, "Polyglot code smell detection for infrastructure as code with glitch," 2023. [Online]. Available: https://arxiv.org/abs/2308.09458

[65] K. C. Thatikonda, "Automating regulatory compliance in cloud-native architectures: A deep learning perspective," *International Research Journal of Modernization in Engineering Technology and Science*, 03 2025. [Online]. Available: https://www.researchgate.net/publication/389550950_AUTOMATING_REGULATORY_COMPLIANCE_IN_CLOUD-NATIVE_ARCHITECTURES_A_DEEP_LEARNING_PERSPECTIVE

[66] Bridgecrew, "Terragoat: Vulnerable by design terraform repository," https://github.com/bridgecrewio/terragoat, 2020, accessed: 2025-04-29.

[67] Tenable, "Kaimonkey: Vulnerable infrastructure as code," https://github.com/tenable/KaiMonkey, 2020, accessed: 2025-04-29.

[68] Bridgecrew, "Checkov: Infrastructure as code static analysis tool," https://www.checkov.io/, 2020, accessed: 2025-04-29.

[69] Tenable, "Terrascan: Detect compliance and security violations across infrastructure as code," https://runterrascan.io/, 2020, accessed: 2025-04-29.

[70] A. Security, "Trivy: All-in-one open source security scanner," https://trivy.dev/, 2020, accessed: 2025-04-29.

[71] MITRE Corporation, "MITRE ATT&CK® Framework," https://attack.mitre.org/, 2023, accessed: 2025-06-02.

[72] Center for Internet Security, "CIS Benchmarks," https://www.cisecurity.org/cis-benchmarks, 2024, accessed: 2025-06-02.

[73] V. B. Parthasarathy, A. Zafar, A. Khan, and A. Shahid, "The ultimate guide to fine-tuning llms from basics to breakthroughs: An exhaustive review of technologies, research, best practices, applied research challenges and opportunities," 2024. [Online]. Available: https://arxiv.org/abs/2408.13296

[74] M. Bavarian, H. Jun, N. Tezak, J. Schulman, C. McLeavey, J. Tworek, and M. Chen, "Efficient training of language models to fill in the middle," 2022. [Online]. Available: https://arxiv.org/abs/2207.14255